

Highlights

Sequence Construction as a Primary Design Factor in Flow-Based Intrusion Detection: A Cross-Year Evaluation on the Locked Shields Datasets

Allard Dijk, Risto Vaarandi, Roland Meier, Mauno Pihelgas

- Controlled evaluation of eight flow-sequence construction strategies for IDS.
- Sequence construction strongly affects detection quality and efficiency.
- 2-Hostname grouping with Longformer achieves AP 0.941 under extreme imbalance.
- Same-year and cross-year performance are anti-correlated ($r = -0.87$).
- LSPR25 dataset is released to support reproducible intrusion-detection research.

Sequence Construction as a Primary Design Factor in Flow-Based Intrusion Detection: A Cross-Year Evaluation on the Locked Shields Datasets

Allard Dijk^{a,*}, Risto Vaarandi^c, Roland Meier^b, Mauno Pihelgas^c

^a*Netherlands Defence Academy (NLDA), Den Helder, The Netherlands*

^b*Armatus Science and Technology, Thun, Switzerland*

^c*Tallinn University of Technology (TalTech), Tallinn, Estonia*

Abstract

In flow-based intrusion detection, how raw network flows are grouped into sequences determines what contextual dependencies a model can learn, yet this representational choice is rarely evaluated as an explicit design factor. This paper presents a controlled empirical study of eight flow sequence construction strategies across the LSPR23, LSPR24, and LSPR25 intrusion detection datasets—three consecutive annual releases derived from the NATO Locked Shields cyber defence exercise—covering 485 configurations and multiple model families (XGBoost, GRU, Transformer).

We evaluate eight grouping strategies, spanning chronological ordering, 5-tuple grouping, IP-centric and IP-pair representations, hostname-centric and hostname-pair representations, and session-based variants, across multiple model families, including XGBoost, GRU, and Transformer encoders. To isolate representational effects, neural model capacity and training procedures are held constant across sequence-model experiments. Across 485 evaluated configurations, the results demonstrate that sequence construction is the primary representational design factor for sequence-aware intrusion detection. On the most challenging and highly imbalanced LSPR25 dataset, a Longformer model using 2-Hostname grouping achieves an Average Precision of 0.9411 despite a positive-class prevalence of only 0.44%, exceeding the XGBoost baseline by +0.090 AP. This advantage widens progressively with dataset difficulty: among Transformer configurations, 2-Hostnames leads on all three years, with its margin growing from +0.006 on LSPR23 to +0.158 on LSPR25. On LSPR23 and LSPR24, detection performance is near-ceiling across model families, leaving limited headroom for structured representations. Cross-year evaluation reveals a strong anti-correlation ($r = -0.87$) between same-year AP and next-year degradation, quantifying how same-year model selection can overestimate deployment readiness. In addition, we release the LSPR25 dataset to support reproducible

*Corresponding author

Email address: ad.dijk@mindef.nl (Allard Dijk)

evaluation under realistic temporal distribution shift.

Keywords: Flow-based intrusion detection, Sequence construction, Network flows, Distribution shift, Benchmark datasets, Transformer models

1. Introduction

A flow-based intrusion detection system (IDS) relies on machine learning (ML) models to distinguish malicious from benign network behaviour using aggregated traffic statistics. Over the past two decades, a wide range of learning paradigms has been explored for this task, including tree-based methods [1, 2, 3], recurrent neural networks [4, 5, 6], graph neural networks [7, 8, 9], and, more recently, Transformer architectures [10, 11, 12]. Despite this architectural diversity, a critical design dimension remains comparatively under-examined: how raw network flows are structured into sequences before learning begins.

In flow-based IDS, sequence construction is not a trivial preprocessing step. Grouping strategies determine which contextual dependencies become observable to a learning algorithm and therefore act as an implicit inductive bias. Whether flows are ordered globally by timestamp, segmented by connection (5-Tuple), aggregated per host, or split into sessions using inactivity thresholds fundamentally changes the temporal and semantic coherence of the resulting sequences. These representational choices influence not only predictive performance but also training efficiency, stability, and robustness under distribution shift.

Recent literature on deep learning for intrusion detection has largely emphasised architectural innovations, particularly the Transformer-based model, while typically relying on a fixed input segmentation or sequence-construction rule [13, 14, 11, 12, 15]. However, it remains unclear whether reported performance gains stem from architectural advances or from representational assumptions embedded in preprocessing. As highlighted in systematisation of knowledge studies on ML for network intrusion detection [16], reproducibility and deployment realism require isolating such design factors under controlled experimental settings. Furthermore, design-tactic analyses of Transformer adaptations for cybersecurity [17] explicitly identify input representation and traffic aggregation as primary variation points, suggesting that sequence construction itself warrants systematic investigation.

This challenge is amplified in realistic cyber defence environments. Public IDS benchmarks are often static and exhibit well-documented evaluation limitations, potentially overstating performance on data drawn from the same benchmark distribution [16, 18]. In contrast, operational scenarios are characterised by severe class imbalance, infrastructural priors, and evolving attack tactics [18]. Under such conditions, preprocessing and representational choices may matter as much as, or more than, purely architectural ones, particularly when assessing generalisation under temporal or cross-dataset distribution shift [16, 18, 19, 20, 21, 12].

Importantly, the datasets considered in this paper are not identical replays with new timestamps; they represent distinct operational scenarios under a shared exercise doctrine. Therefore, cross-year performance degradation likely reflects a combination of attack evolution and broader traffic distribution shifts that cannot easily be disentangled. In this work, we do not attempt to disentangle these factors causally. Instead, we treat cross-year evaluation as an operational proxy for deployment under realistic temporal distribution shift, focusing on whether certain sequence construction strategies yield representations that are inherently more robust to such shifts. In practice, cross-year degradation may manifest as (i) shifts in benign-malicious class balance, (ii) changes in sequence length distributions induced by evolving infrastructure scale and grouping density, and (iii) altered ranking behaviour under fixed decision thresholds. These effects are quantified in Section 6 to clarify how representational robustness interacts with structural scenario evolution. We additionally examine the interaction between grouping strategies and positional encodings, not as an architectural competition, but as a secondary factor that may modulate representational coherence.

1.1. Research Questions and Contributions of the Paper

To systematically analyse sequence construction as a primary design consideration in flow-based intrusion detection, we investigate the following research questions:

- **RQ1:** How does flow sequence construction influence detection performance under class imbalance, as measured by both threshold-dependent (F_1) and threshold-free metrics (e.g., AP)?
- **RQ2:** How do different grouping strategies affect robustness under cross-year distribution shift (LSPR23→LSPR24→LSPR25)?
- **RQ3:** To what extent do positional encodings interact with the inherent structural properties of constructed sequences?
- **RQ4:** What is the trade-off between detection performance and computational efficiency (e.g., sequence count, per-epoch training time) across grouping strategies?

◆ To address these questions, we conduct a controlled empirical study of sequence construction in flow-based intrusion detection. We evaluate eight grouping strategies across three consecutive Locked Shields datasets (LSPR23–LSPR25), comparing chronological ordering, connection-level grouping, host-centric representations, host-pair grouping, and session-aware variants. The primary contribution of this paper is this controlled empirical study of sequence construction across three annual Locked Shields datasets. The secondary contribution is the public release of the LSPR25 dataset, extending the annually updated LSPR series [22, 23, 24]. Unlike static benchmarks, this evolving dataset series supports reproducible evaluation under changing network topologies and

attack behaviours, enabling systematic study of representation-sensitive intrusion detection under realistic temporal distribution shift. To isolate representational effects, model capacity and optimisation settings are held fixed across sequence-model experiments, enabling systematic analysis of detection performance, robustness under distribution shift, and computational efficiency.

The remainder of this paper is organised as follows. Section 2 reviews prior work on flow-based intrusion detection and representation design. Section 3 describes the Locked Shields datasets and data preprocessing. Section 4 presents the formal definition of sequence construction strategies. Section 5 describes the experimental setup, including model architectures, training procedures, and evaluation methodology. Section 6 reports empirical findings, Section 7 discusses the implications, and Section 8 concludes the paper.

2. Related Work

In flow-based intrusion detection, how network events are aggregated into flows and subsequently organised into sequences strongly shapes what a model can learn from those events [25, 26]. Existing work is therefore relevant along three connected dimensions: how flow records are constructed and featurised, how traffic is represented for sequence-aware learning, and how reported performance behaves under realistic distribution shift. This section reviews those research directions and then highlights the research gap addressed in this paper.

2.1. Flow construction, feature extraction, and benchmark artefacts

Flow-based IDS pipelines begin with measurement: packets are aggregated into flow records and exported using NetFlow/IPFIX-style systems. As Hofstede et al. show, a *flow* is a measurement construct rather than a fixed natural unit, since exporter implementation, sampling, cache behaviour, and especially active and inactive (idle) timeouts determine how much traffic is represented by each exported record [25]. These design choices directly affect both the number of produced flow records and their temporal boundaries, and therefore influence any downstream sequence construction built on top of them.

Many widely used IDS benchmarks and research pipelines derive per-flow features with CICFlowMeter or related ISCX/CIC tooling [27, 28, 29, 20, 30, 31]. The paper introducing the CIC-IDS2017 dataset explicitly presents CICFlowMeter-based feature extraction as a core part of dataset construction [27]. Subsequent analyses have shown that such preprocessing choices are not merely technical details: Engelen et al. revisit CICIDS2017 and demonstrate that issues in flow generation and labelling can materially affect downstream evaluation [28], while Sarhan et al. show that feature-set selection influences both explainability and generalisability in ML-based network intrusion detection [20]. More broadly, security-ML work has repeatedly warned that dataset construction and preprocessing pipelines can introduce structural artefacts and shortcuts that inflate apparent performance without improving operational robustness [32, 33]. For

the present study, these observations motivate treating flow extraction parameters and sequence construction rules as substantive modelling choices rather than as implementation trivia.

2.2. Sequence representations and model families for flow-based IDS

Sequence construction determines which dependencies are exposed to a learner. Common choices include global chronological chunking, connection-centric grouping (e.g., 5-tuple dialogues), endpoint-centric grouping, host-pair grouping, and session-aware variants that split activity using inactivity thresholds [25, 34]. These alternatives embody different inductive biases: global chronology preserves overall temporal order but interleaves unrelated conversations; connection-centric grouping yields semantically narrow exchanges; host-centric grouping emphasises longer-lived entity behaviour; and session-based splitting attempts to recover coherent episodes of activity.

Prior IDS work contains many examples of sequence-like structuring, but in most cases the sequence-construction rule is fixed *a priori* rather than isolated as the primary experimental variable [26, 13, 14, 11, 15, 12]. For example, Nguyen and Watabe represent temporally adjacent flow sequences as token sequences for a BERT-based IDS, but the sequence construction itself is treated as given rather than systematically varied [13]. Han et al. propose a time-aware Transformer that relies on a predefined traffic representation combining local sequence statistics with temporal ordering [14]. Long et al. and Xi et al. likewise investigate Transformer-based IDS designs under fixed traffic segmentation choices, focusing on architectural refinements rather than on systematic comparison of grouping strategies [11, 15]. A recent paper proposing the Flow-Transformer framework makes this point particularly clearly: Manocchio et al. show that apparently secondary choices such as input encoding, Transformer backbone, and classification head can materially affect performance, with some components dominating outcomes in practice [12]. This is directly relevant here, because if such seemingly minor design choices already have a significant impact, then grouping rules that alter sequence length, coherence, and entity context should also be evaluated explicitly under controlled settings.

Beyond Transformers, sequence-aware IDS has employed recurrent and graph-based models. Recurrent architectures emphasise temporal locality and remain competitive for ordered traffic data, especially when short-range dependencies dominate [35, 4, 5]. Clausen et al. argue that many attacks are contextual rather than globally anomalous, and show that preserving local behavioural structure can reduce false positives for certain remote-access scenarios [36]. Graph neural networks provide a different route to contextualisation by representing communication as evolving host-host graphs, thereby making relationships between entities explicit rather than implicit in a sequence [7, 8, 9]. As Zhong et al. note in their survey, graph-based methods can capture rich relational structure, but they also introduce additional modelling assumptions through graph construction, temporal graph design, and neighbourhood aggregation [37]. Taken together, these studies indicate that model family matters, but also that the representation presented to the model remains a first-order design decision.

2.3. Generalisability under preprocessing variation and dataset shift

A second recurring theme in the literature is that strong same-dataset performance often fails to transfer across datasets or operational settings. Apruzzese et al. provide a pragmatic assessment of ML-based network intrusion detection and argue that deployment realism depends not only on architecture but also on feature selection, evaluation design, and operational assumptions [16]. Their cross-evaluation study further shows that models trained and tuned on one intrusion-detection benchmark can degrade substantially when evaluated on others [38]. Cantone et al. reach a similar conclusion in a recent cross-dataset generalisation study, reporting that many high-performing models are strongly dependent on benchmark-specific structure rather than robust intrusion signatures [19].

This concern extends directly to preprocessing and representation. Sarhan et al. show that standard feature-set choices influence the generalisability and explainability of ML-based IDS models [20], while Manocchio et al. demonstrate empirically that preprocessing decisions can substantially change performance in network intrusion detection pipelines [21]. These results reinforce the view that evaluation of sequence models should not interpret architecture in isolation from upstream representation choices.

At the same time, most public IDS benchmarks remain static. The Locked Shields dataset series differs from that setting by providing annually evolving exercise-derived corpora with a consistent labelling approach and operational provenance [23, 24]. Reviews of IDS dataset construction have likewise emphasised the need for transparent tooling, documented flow extraction, and accessible benchmark releases to support reproducible research [39]. This creates an opportunity to examine whether sequence-construction choices remain stable under realistic cross-year variation rather than only within a single fixed benchmark.

2.4. Research gap and connection to our contribution

Across these research directions, a consistent gap remains. Existing work has shown that preprocessing, feature extraction, and model architecture all matter, and several recent papers have explored sequence-aware learning for flow-based IDS. However, sequence construction itself is rarely isolated as the primary experimental variable under controlled model capacity and realistic, evolving datasets. Most prior studies either propose a new architecture under a fixed grouping rule or report results on static benchmarks known to contain extraction artefacts [16, 33]. Our study addresses this gap through a controlled comparison of eight grouping strategies on the annually evolving LSPR23–LSPR25 datasets, while holding neural model capacity and training procedures fixed across sequence-model experiments.

3. Dataset

3.1. Locked Shields Dataset Characteristics

The datasets used in this study originate from the NATO Cooperative Cyber Defence Centre of Excellence (CCDCOE) Locked Shields live-fire cyber defence exercise. Locked Shields is a large-scale cyber defence exercise in which blue teams defend realistic national-scale infrastructures against a professional red team operating under defined objectives and constraints. The datasets LSPR23, LSPR24, and LSPR25 correspond to three consecutive annual Partners Run editions of this exercise and share a consistent labelling methodology and comparable data collection pipeline [23, 24].

The annually evolving nature of this dataset series is particularly valuable for the present study because it enables evaluation under realistic year-to-year variation rather than only within a single static benchmark. The LSPR23 dataset paper presents Locked Shields traffic as large-scale live-fire exercise data with well-documented attacker actions and labels, intended to better approximate defender-attacker dynamics than many replay-based public benchmarks [23]. The LSPR24 work further emphasises reproducibility and operational relevance in support of blue-team automation research [24]. More broadly, reviews of intrusion-detection dataset construction have highlighted the importance of transparent tooling, documented flow extraction, and accessible benchmark releases for reproducible evaluation [39].

Cross-year distribution shift. In this study, we use *distribution shift* to denote cross-year differences between LSPR23, LSPR24, and LSPR25. These differences arise from structural scenario evolution, including (i) changes in infrastructure scale and topology (number of hosts, services, and network segments), (ii) variation in attack narratives and tactics across annual exercise designs, (iii) shifts in defender behaviour and operational responses, and (iv) changes in class balance between benign and malicious activity.

These factors alter both the distribution of flows and the induced sequence structures (e.g., sequence length distributions and grouping density). They may also affect the relationship between observed traffic patterns and the malicious label, but this study does not attempt to isolate those components separately. Cross-year evaluation therefore probes model robustness under realistic operational evolution rather than under i.i.d. assumptions.

Infrastructure realism. The exercise infrastructure combines heterogeneous information technology (IT) and operational technology (OT) components. The topology includes enterprise services (e.g., web, mail, DNS and related back-end systems), ISP infrastructure with BGP routing, firewalls, VPN gateways, and redundant routers and switches. In addition, specialised systems such as SCADA/ICS for power plant control, SWIFT banking services, military border guard systems, satellite communication, and 5G components are deployed. Multiple operating systems and service stacks are present. Each blue team defends its own segmented infrastructure while interacting with shared external and

ISP components. This heterogeneity reflects the structural complexity typically encountered in operationally deployed enterprise and governmental networks.

Traffic composition and scale. Network traffic was recorded via ERSpan across participating systems, subject to documented exercise downtime windows outside active operational hours. The datasets capture activity from defenders (blue teams), attackers (red teams), organisers (green/white teams), and user simulation infrastructure, thereby including both benign background behaviour and adversarial interactions. The Partners Run editions span two operational days: a familiarisation phase without active attacks, followed by an attack phase during which the red team launches coordinated multi-stage operations. Across LSPR23-LSPR25, the datasets range from over 16 million to over 30 million network flows per year, each enriched with over 80 features derived from CICFlowMeter and exercise metadata. This scale is comparable to or larger than many widely used public intrusion detection benchmarks and enables statistically meaningful evaluation under severe class imbalance.

Attack diversity and operational behaviour. The red team operates in specialised subteams targeting client networks, web services, and specialised infrastructure components. Documented attack goals include privilege escalation, system compromise, data theft, website defacement, and non-destructive operations. Observed methods include remote code execution, credential abuse (default, stolen, and red-team credentials), remote desktop access, command-and-control activity, BGP manipulation, and backdoor implantation. High-level attack narratives are provided and linked to structured metadata (including MITRE ATT&CK classifications) [23]. Attacks are executed by red-team operators within an interactive exercise setting, and blue teams actively respond during the event. Consequently, the captured traffic reflects attacker-defender dynamics under operational constraints rather than isolated or replayed attack traces.

Ground truth and labelling consistency. Flow-level labels are derived from official exercise role metadata and red-team infrastructure identifiers, as formally defined in the LSPR23 dataset paper [23]. A flow is labelled malicious if its source or destination IP corresponds to red-team infrastructure during the exercise. The same labelling procedure is applied consistently across LSPR23, LSPR24, and LSPR25, supporting methodological comparability in cross-year experiments. Known limitations, such as the inability to label certain stepping-stone attacks, are transparently documented in the dataset publications. In addition to binary flow labels, the datasets provide enriched contextual metadata, including network segments, host roles, and high-level attack information, enabling fine-grained analysis beyond binary classification. Although the labelling procedure is derived from official exercise role metadata and infrastructure identifiers, we do not assume perfect label purity. Known limitations (e.g., stepping-stone attacks) may introduce conservative false negatives, but the labelling mechanism is consistent and reproducible across all datasets.

Class imbalance and labelling granularity. Labels are assigned at the *flow level*: each individual flow record receives a binary benign/malicious label based on whether its source or destination IP belongs to red-team infrastructure. This flow-level granularity propagates through all sequence construction strategies and evaluation metrics. Class imbalance varies substantially across datasets and is a defining characteristic of the evaluation. In the OP representation, the positive-class prevalence (π) is approximately 30.0% for LSPR23, 7.1% for LSPR24, and 2.2% for LSPR25, corresponding to benign-to-malicious ratios of approximately 2.3:1, 13:1, and 45:1, respectively. Under entity-based grouping strategies, induced π values change further due to redistribution of flows across sequences; for instance, 2-Hostname grouping yields $\pi \approx 0.93\%$, 0.43% , and 0.44% on LSPR23, LSPR24, and LSPR25, respectively. The complete per-strategy π values are reported in Table 8. This severe and strategy-dependent imbalance motivates the use of Average Precision as the primary metric, as it directly reflects ranking quality among the positive class without requiring a fixed decision threshold.

Operational representativeness. Unlike static replay-based benchmarks, the Locked Shields datasets originate from a live operational cyber defence scenario involving heterogeneous infrastructure, multi-team interaction, and human-driven attack campaigns. The annually evolving LSPR23–LSPR25 series introduces scenario-level variation in topology, deployed services, and attack tactics, enabling evaluation under realistic distribution shift.

3.2. Data Preparation and Preprocessing

For each dataset, flows were extracted and labelled using the available information sources as described in [23]. Each raw flow record includes standard metadata and traffic statistics such as timestamps, source and destination IP addresses, ports, transport protocol, and byte and packet counts. Raw identity-like identifiers, including IP addresses and hostnames, are not used directly as model inputs. Instead, learning is performed on the derived numeric flow features listed in Appendix A, together with coarse non-identity indicators such as `External_src`, `External_dst`, and `Int/Ext Dst IP`. These features encode only broad network-location information derived from the exercise infrastructure (e.g., internal versus external network segments) and do not reveal attacker identity; we additionally verified through feature-level leakage checks that they do not deterministically encode the malicious label. A uniform preprocessing pipeline—covering flow extraction, labelling, feature selection, normalisation, and train/validation splitting—was applied across LSPR23–LSPR25 to ensure that observed performance differences reflect experimental factors rather than inconsistencies in data handling.

4. Methodology

4.1. Sequence Construction Strategies

The representation of network data significantly influences the performance of sequence models in intrusion detection tasks. Different sequence grouping strategies capture distinct aspects of network behaviour and therefore induce different structural priors for learning. In this work, each strategy is specified by (i) its intuition and rationale and (ii) its formal grouping definition. Rather than presenting a separate pseudocode block for each strategy, we factorise sequence construction into a single generic procedure and instantiate it through strategy-specific grouping functions.

Definition of Sequence Structure. In this study, we use the term *sequence structure* to denote the degree to which a grouping strategy preserves coherent behavioural locality within a sequence. Specifically, a sequence is considered more structured if flows within it share a persistent logical context (e.g., common host, IP pair, or session) and/or represent temporally contiguous activity of that context. Conversely, strategies that aggregate heterogeneous flows solely by global chronology—such as Original Position (OP)—provide minimal entity-level coherence and are therefore treated as weakly structured baselines.

Basic notation. Let $\mathcal{F}$ denote the full set of extracted flow records for a dataset. Let $[f_1, \dots, f_m]$ denote a sequence of flows. If $X = [f_1, \dots, f_m]$ is a sequence of flows, then $Y \preceq X$ denotes that Y is a contiguous subsequence of X , i.e., $Y = [f_i, \dots, f_j]$ for some $1 \leq i \leq j \leq m$. If $X = [f_1, \dots, f_m]$ and $Y = [f'_1, \dots, f'_k]$ are sequences of flows, then $X \circ Y$ denotes their concatenation:

$$X \circ Y = [f_1, \dots, f_m, f'_1, \dots, f'_k].$$

Throughout this methodology, $s(f)$ denotes the recorded flow start timestamp (`mTimestampStart`).

Chunking convention (partial chunks included). For any ordered flow sequence $X = [f_1, \dots, f_m]$, the chunking operator $\text{Chunk}_L(X)$ partitions X into $\lceil m/L \rceil$ contiguous, non-overlapping subsequences of at most L elements each. Formally, the k -th chunk is

$$C_k = [f_{(k-1)L+1}, f_{(k-1)L+2}, \dots, f_{\min(kL, m)}], \quad k = 1, \dots, \lceil m/L \rceil.$$

If $m \bmod L \neq 0$, the final chunk has length $m \bmod L$ and is included, i.e., no elements are discarded. In the implemented preprocessing pipeline used for all experiments, the chunk size was fixed to $L = 128$ flows.

Generic sequence-construction framework. All evaluated sequence strategies can be expressed as instances of a common procedure: (i) assign each flow to one or more grouping keys, (ii) collect flows per key, (iii) sort each group chronologically, (iv) optionally split each ordered group into sessions using an inactivity threshold Δ , and (v) partition each resulting ordered list into chunks of length at most L .

Formally, let Φ denote a mapping of flows to a set of grouping keys assigned to that flow $f \in \mathcal{F}$. Depending on the strategy, $\Phi(f)$ may contain one key (e.g., $\Phi_{\text{OP}}(f)$, $\Phi_{5\text{T}}(f)$, $\Phi_{2\text{IP}}(f)$, $\Phi_{2\text{H}}(f)$) or multiple keys (e.g., $\Phi_{1\text{IP}}(f)$ and $\Phi_{1\text{H}}(f)$), where a flow is assigned to both endpoints). For each key u , define the corresponding subset of flows

$$H_u = \{ f \in \mathcal{F} : u \in \Phi(f) \}.$$

Let G_u denote the sequence obtained by sorting the elements of H_u according to the order induced by the timestamp function s (defined above), with ties broken by dataset row order. We write $G_u[i]$ for the i -th element of this sequence, so that $s(G_u[i]) \leq s(G_u[j])$ for all $1 \leq i < j \leq |H_u|$.

Shared inactivity threshold for session-based strategies. For all session-based strategies, session boundaries are defined using a fixed inactivity threshold of $\Delta = 300$ seconds applied to consecutive recorded start timestamps within each ordered group. Here, *session* denotes a higher-level segment over already extracted flow records, not an individual TCP session. TCP termination is therefore handled at flow-extraction time; the session-based strategies defined here may still group multiple distinct TCP flows into the same ordered session segment when they share a grouping key and fall within the inactivity threshold Δ .

If session splitting is enabled, G_u is partitioned into k non-empty contiguous subsequences $G_{u,1}, \dots, G_{u,k}$ such that $G_{u,1} \circ \dots \circ G_{u,k} = G_u$. Session boundaries are placed at every position where consecutive timestamps exceed the threshold: within each segment $G_{u,r}$, consecutive elements satisfy

$$s(G_{u,r}[i]) - s(G_{u,r}[i-1]) \leq \Delta \quad \text{for all } 2 \leq i \leq |G_{u,r}|,$$

and between consecutive segments,

$$s(G_{u,r+1}[1]) - s(G_{u,r}[|G_{u,r}|]) > \Delta \quad \text{for all } 1 \leq r < k.$$

In other words, each $G_{u,r}$ is a maximal contiguous subsequence of G_u whose consecutive timestamp gaps do not exceed Δ . If session splitting is not enabled, G_u is used directly. Finally, each resulting ordered list is transformed into sequences using the chunking operator $\text{Chunk}_L(G_u)$.

4.1.1. Original Position (OP)

Intuition and Rationale. The Original Position strategy sequences flows according to their global timestamps, using the chronological order provided in the

raw dataset. This corresponds to a common chronological baseline in recent sequence-based IDS studies, where packets or flows are organised in temporal order or into temporally adjacent windows before being passed to the model, without additional entity-based grouping [13, 14, 40]. By chunking flows directly from this global temporal stream, the method preserves the overall timeline of network activity across all hosts. It therefore serves as a reference point for comparison with alternative grouping methods that introduce additional structural or contextual segmentation. In our experiments, flows are globally sorted once by their recorded start timestamps prior to sequence construction, resulting in deterministic offline ordering. We do not simulate streaming ingestion; rather, OP represents a batch chronological baseline. Ties in timestamp are broken by the original dataset row order, ensuring a fully deterministic and reproducible sequence.

Formal Mathematical Definition. Each strategy below defines a specific instance of the generic grouping function Φ introduced above. The Original Position strategy is obtained by assigning every flow to a single global key:

$$\Phi_{\text{OP}}(f) = \{1\}.$$

Thus, all flows are assigned to one common ordered stream, which is transformed into sequences according to the generic sequence-construction framework. Session splitting is not applied for OP.

4.1.2. 5-Tuple Sequences

Intuition and Rationale. The 5-Tuple strategy extends the traditional concept of network flow grouping by organising sequences according to the canonical 5-tuple—source IP, destination IP, source port, destination port, and transport protocol. While the 5-tuple is typically used to define a unidirectional flow record in raw network traffic, this strategy first applies a deterministic canonicalisation by independently lexicographically sorting the unordered IP pair $\{\text{srcIP}, \text{dstIP}\}$ and the unordered port pair $\{\text{srcPort}, \text{dstPort}\}$ (after casting ports to strings), and then concatenating the results with the transport protocol to obtain a direction-agnostic bidirectional key. By doing so, it elevates the analysis from individual directed flow records to broader interaction-level groupings, providing a higher-level abstraction that may reveal behavioural patterns or anomalies spanning the communication context between hosts and services.

◆ This method thus bridges the detail of packet-level observations and the abstraction needed for sequence modelling in anomaly-detection tasks. We did not apply additional session splitting to 5-Tuple groups because preliminary tests indicated that doing so typically produced very short sequences, often of length ≤ 2 , thereby providing little additional temporal context for the model to exploit.

Formal Mathematical Definition. Let $\pi(f) = (a, b, c, d, \text{proto})$ denote the raw 5-tuple of flow f , where

$$a = \text{srcIP}(f), \quad b = \text{dstIP}(f), \quad c = \text{srcPort}(f), \quad d = \text{dstPort}(f).$$

Canonicalisation is performed component-wise using lexicographic (string) ordering, matching our preprocessing implementation¹. Let a, b be IP address strings and let $\text{str}(c), \text{str}(d)$ be the corresponding port numbers cast to strings. Define

$$(a_{\min}, a_{\max}) = \text{sort}_{\text{lex}}\{a, b\}, \quad (c_{\min}, c_{\max}) = \text{sort}_{\text{lex}}\{\text{str}(c), \text{str}(d)\},$$

and set the canonical bidirectional key

$$\psi(\pi(f)) = (a_{\min}, a_{\max}, c_{\min}, c_{\max}, \text{proto}).$$

Within the generic sequence-construction framework introduced above, the 5-Tuple strategy assigns each flow to the canonicalised bidirectional 5-tuple key

$$\Phi_{5T}(f) = \{\psi(\pi(f))\}.$$

The resulting ordered groups are then transformed into sequences according to the generic sequence-construction framework. Session splitting is not applied for this strategy.

4.1.3. 1-IP Sequences

Intuition and Rationale. The 1-IP strategy adopts a host-centric view by assigning each flow twice—once to its source IP address and once to its destination IP address. This produces longer per-host sequences that may expose host-level behavioural context, including multi-step activity associated with the same endpoint. Such broader context may assist the detection of behaviours such as lateral movement or repeated interaction patterns. However, because all activity associated with an IP address is aggregated into one chronological stream, unrelated events occurring over longer periods may also be concatenated, potentially diluting local context.

Formal Mathematical Definition. Within the generic sequence-construction framework introduced above, the 1-IP strategy is defined by assigning each flow to the IP addresses of its two participating endpoints. Formally, let

$$a = \text{srcIP}(f), \quad b = \text{dstIP}(f).$$

The grouping assignment function is then defined by

$$\Phi_{1IP}(f) = \{a, b\}.$$

The resulting ordered groups are transformed into sequences according to the generic sequence-construction framework. Session splitting is not applied for this strategy.

¹IP addresses are compared as ASCII strings (e.g., $192.168.1.10 < 192.168.1.9$ lexicographically). Port numbers are likewise compared as their string representations. This convention is deterministic and matches the implementation but does not correspond to numerical IP ordering.

4.1.4. 1-IP-Session Sequences

Intuition and Rationale. The session-based 1-IP strategy refines the host-centric 1-IP representation by partitioning each IP-centred activity stream into temporally localised sessions separated by inactivity gaps. This reduces the tendency of the full 1-IP strategy to merge unrelated events occurring at widely separated times, and therefore yields sequences that more closely reflect coherent episodes of activity. In practice, this may improve the relevance of the resulting sequences for anomaly detection by preserving shorter-term behavioural locality. The trade-off is that sessionisation increases the number of induced sequences, which may reduce batching efficiency and increase training time.

Formal Mathematical Definition. Within the generic sequence-construction framework introduced above, the session-based 1-IP strategy assigns each flow to the two grouping keys corresponding to its source and destination IP addresses. Formally, let

$$a = \text{srcIP}(f), \quad b = \text{dstIP}(f).$$

The grouping assignment function is then defined by

$$\Phi_{\text{SesIP}}(f) = \{a, b\}.$$

Thus, each flow contributes to the ordered groups associated with both participating endpoints. The resulting groups are then sessionised using the shared inactivity threshold Δ and transformed into sequences according to the generic sequence-construction framework.

4.1.5. 2-IP Sequences

Intuition and Rationale. The 2-IP strategy groups flows according to unordered pairs of source and destination IP addresses, while disregarding ports and protocols as grouping keys. Each resulting sequence therefore represents the chronological interaction history between two endpoints and can capture multi-step behaviour spanning multiple services or ports within the same host pair. This may be advantageous for identifying attacks that unfold through repeated exchanges between the same two endpoints. However, because the representation is restricted to individual host pairs, broader activity involving multiple interacting hosts may remain fragmented. We did not apply additional session splitting to 2-IP groups because, as in the 5-Tuple case, preliminary analysis indicated that further inactivity-based splitting typically produced very short sequences and therefore contributed little additional temporal context for the model to exploit.

Formal Mathematical Definition. Within the generic sequence-construction framework introduced above, the 2-IP strategy assigns each flow to a single grouping key corresponding to the unordered pair of its communicating IP addresses. Formally, let

$$a = \text{srcIP}(f), \quad b = \text{dstIP}(f).$$

To obtain a direction-agnostic representation, define the canonicalised IP-pair key

$$\chi(a, b) = \text{sort}_{\text{lex}}\{a, b\} = (p, q),$$

where $p \leq q$ under lexicographic ordering. The grouping assignment function is then defined by

$$\Phi_{2\text{IP}}(f) = \{\chi(\text{srcIP}(f), \text{dstIP}(f))\}.$$

Thus, each flow is assigned to exactly one unordered host-pair key, and the resulting ordered groups are transformed into sequences according to the generic sequence-construction framework. Session splitting is not applied for this strategy.

4.1.6. Single Hostname-Based Sequences

Intuition and Rationale. The Single Hostname strategy organises network flows according to dataset-provided logical endpoint identifiers rather than network-layer addresses. For benign endpoints, the grouping key is the endpoint's *Expoid*, i.e., the identifier recorded in the Locked Shields Exercise Portal metadata; for non-benign endpoints, the raw IP address is used as fallback. This design abstracts from transient IP assignments for benign infrastructure and groups flows by persistent exercise-asset identity rather than by address alone. As a result, flows that would otherwise be separated by IP churn or address reassignment can still be associated with the same logical benign endpoint. For non-benign endpoints, we deliberately avoid using Expoid-style identifiers, even where such metadata may be present, because those identifiers would not be realistically available to defenders during operation. Grouping therefore remains IP-based for adversary-controlled, external, or otherwise non-benign endpoints in order to preserve deployment realism and avoid metadata leakage.

Endpoint-Identity Mapping. Endpoint identifiers are obtained directly from the Locked Shields dataset fields `Expoid_src` and `Expoid_dst`, as used in the pre-processing pipeline that generates the experimental sequence data. Here, *Expoid* denotes the identifier associated with an endpoint in the Locked Shields Exercise Portal (EXPO), the central system used by exercise organisers to collect and manage infrastructure and role metadata from multiple sources. These identifiers are therefore dataset-provided exercise metadata, not values inferred via reverse DNS, passive DNS, or hostname-resolution techniques. No DNS-based consolidation or post-hoc alias resolution is performed.

For all hostname-based strategies, each endpoint is mapped to a logical identifier according to the following rule:

$$\rho_{\text{src}}(f) = \begin{cases} \text{SrcExpoid}(f), & \text{if } \text{SrcLabel}(f) = 0, \\ \text{SrcIP}(f), & \text{otherwise,} \end{cases}$$

$$\rho_{\text{dst}}(f) = \begin{cases} \text{DstExpoid}(f), & \text{if } \text{DstLabel}(f) = 0, \\ \text{DstIP}(f), & \text{otherwise.} \end{cases}$$

Formal Mathematical Definition. Within the generic sequence-construction framework introduced above, the Single Hostname strategy assigns each flow to the two logical endpoint identifiers corresponding to its source and destination endpoints. The grouping assignment function is therefore defined by

$$\Phi_{\text{Host}}(f) = \{\rho_{\text{src}}(f), \rho_{\text{dst}}(f)\}.$$

Thus, each flow contributes to the ordered groups associated with both participating endpoints. The resulting groups are transformed into sequences according to the generic sequence-construction framework. Session splitting is not applied for this strategy.

4.1.7. Single Hostname Session-based Sequences (1-Hostname-session)

Intuition and Rationale. This strategy refines the Single Hostname representation by partitioning each logical endpoint stream into sessions separated by a fixed inactivity threshold. The resulting sequences therefore reflect temporally localised activity associated with the same logical entity, reducing the blending of unrelated behaviours that may arise when flows are aggregated over longer periods. As with the non-sessionised hostname-based variant, endpoints without benign Expoid-based identifiers are represented using their IP addresses, ensuring coverage for non-benign and external entities.

Formal Mathematical Definition. Using the shared endpoint-identity mapping defined above, the session-based hostname strategy assigns each flow to the two logical endpoint identifiers corresponding to its source and destination endpoints. The grouping assignment function is therefore defined by

$$\Phi_{\text{SesH}}(f) = \{\rho_{\text{src}}(f), \rho_{\text{dst}}(f)\}.$$

Thus, each flow contributes to the ordered groups associated with both participating endpoints. The resulting groups are then sessionised using the shared inactivity threshold Δ and transformed into sequences according to the generic sequence-construction framework.

4.1.8. 2-Hostnames-Based Sequences (2-Hostnames)

Intuition and Rationale. The 2-Hostnames strategy forms sequences by grouping flows according to unordered pairs of logical endpoint identifiers, with IP-address fallback where benign Expoid-based identifiers are unavailable. This representation aggregates bidirectional communication between two entities independently of network-layer direction and can therefore capture more persistent interaction patterns than representations based strictly on directed flows. It provides an entity-level analogue of the 2-IP strategy by abstracting from transient addressing for benign infrastructure while preserving deployment realism for non-benign and external endpoints. In dual-stack environments where hosts use multiple IPv4 and IPv6 addresses, hostname-based grouping consolidates all traffic between two logical assets into a single sequence, whereas IP-based grouping would fragment it across disjoint groups.

Formal Mathematical Definition. Using the shared endpoint-identity mapping defined above, the 2-Hostnames strategy assigns each flow to a single grouping key corresponding to the unordered pair of its communicating entities. To obtain a direction-agnostic representation, define the canonicalised entity-pair key

$$\eta(\rho_{\text{src}}(f), \rho_{\text{dst}}(f)) = \text{sort}_{\text{lex}}\{\rho_{\text{src}}(f), \rho_{\text{dst}}(f)\} = (e_{\min}, e_{\max}),$$

where $e_{\min} \leq e_{\max}$ under lexicographic ordering. The grouping assignment function is then defined by

$$\Phi_{2\text{H}}(f) = \{\eta(\rho_{\text{src}}(f), \rho_{\text{dst}}(f))\}.$$

Thus, each flow is assigned to exactly one unordered logical entity-pair key, and the resulting ordered groups are transformed into sequences according to the generic sequence-construction framework. Session splitting is not applied for this strategy.

4.2. Positional Embeddings

Transformer architectures inherently lack an understanding of sequential order. Positional embeddings are therefore employed to inject sequence information into the input data. To examine the impact of positional embeddings on transformer-based anomaly detection in network flows, we consider three types: *sinusoidal*, *Rotary Position Embeddings (RoPE)*, and a baseline configuration employing *no positional embeddings*.

The sinusoidal positional embedding, introduced by Vaswani et al. [41], encodes positional information using sinusoidal functions of varying frequencies. This approach ensures that relative positions between tokens are systematically encoded, facilitating the model’s comprehension of token ordering without learned parameters.

Rotary Position Embeddings (RoPE), proposed by Su et al. [42], integrate positional information directly into the query and key matrices through rotational transformations in the embedding space. This method enables relative positional awareness in self-attention layers, offering robustness in sequence modelling tasks by explicitly encoding relative distances.

Lastly, we include experiments without positional embeddings to evaluate whether positional context contributes under different sequence construction strategies, as prior work has reported mixed findings in flow-based settings [13].

4.3. Illustrative Example of Sequence Grouping

To clarify the differences between the evaluated sequence-grouping strategies, we apply them to a small synthetic set of network flows shown in Table 1. For readability, we report representative example groups and resulting sequences rather than exhaustively listing every constructed sequence. For didactic purposes, the session-based examples below use an illustrative threshold of $\Delta = 60$ s so that session boundaries are visible in the toy example; in the actual pre-processing pipeline used for all experiments, the threshold was fixed at $\Delta = 300$ s.

Flow	Time (s)	Hostnames	IP	IP Ver.	Ports	Proto
A	0	M1→M2	10.0.0.1→10.0.0.2	v4	1234→80	TCP
B	5	M3→M1	fe80::3→fe80::1	v6	443→5678	TCP
C	10	M2→M1	10.0.0.2→10.0.0.1	v4	80→1234	TCP
D	15	M4→M5	fe80::4→fe80::5	v6	53→5353	UDP
E	20	M1→M2	10.0.0.1→10.0.0.2	v4	1235→80	TCP
F	120	M3→M4	fe80::3→fe80::4	v6	22→2222	TCP
G	125	M2→M3	10.0.0.2→10.0.0.3	v4	80→443	TCP
H	130	M5→M1	fe80::5→fe80::1	v6	8080→8081	TCP
I	400	M1→M3	10.0.0.1→10.0.0.3	v4	8000→80	TCP
J	405	M4→M2	fe80::4→fe80::2	v6	161→162	UDP

Table 1: Illustrative sample of 10 network flows used to demonstrate the effect of different sequence-grouping strategies.

Original Position (OP) All flows are globally ordered by time and chunked into contiguous blocks of size $L = 4$:

$$S_1^{\text{OP}} = [A, B, C, D], \quad S_2^{\text{OP}} = [E, F, G, H], \quad S_3^{\text{OP}} = [I, J].$$

5-Tuple (5-T) Flows are grouped by canonicalised bidirectional 5-tuples. For example, the canonical key

$$\mathbf{p}_1 = (10.0.0.1, 10.0.0.2, 80, 1234, \text{TCP})$$

collects the bidirectional pair of flows A and C , yielding

$$S_{\mathbf{p}_1,1}^{5\text{T}} = [A, C].$$

Similarly, the canonical key for flow B yields $S_{\mathbf{p}_2,1}^{5\text{T}} = [B]$, and the key for flow D yields $S_{\mathbf{p}_3,1}^{5\text{T}} = [D]$.

1-IP Flows are grouped by individual IP address, with each flow contributing once to its source IP group and once to its destination IP group. For example,

$$G_{10.0.0.1} = [A, C, E, I] \Rightarrow S_{10.0.0.1,1}^{1\text{IP}} = [A, C, E, I],$$

$$G_{10.0.0.2} = [A, C, E, G, J] \Rightarrow S_{10.0.0.2,1}^{1\text{IP}} = [A, C, E, G], \quad S_{10.0.0.2,2}^{1\text{IP}} = [J],$$

and

$$G_{10.0.0.3} = [G, I] \Rightarrow S_{10.0.0.3,1}^{1\text{IP}} = [G, I].$$

1-IP-Session (SesIP) Flows are first grouped by IP address and then split into sessions when the gap between consecutive start times-tamps exceeds $\Delta = 60$ s. For example,

$$G_{10.0.0.1} = [A, C, E, I]$$

has timestamp gaps 10, 10, 380, so it splits into two sessions:

$$G_{10.0.0.1,1} = [A, C, E], \quad G_{10.0.0.1,2} = [I].$$

Thus,

$$S_{10.0.0.1,1,1}^{\text{SesIP}} = [A, C, E], \quad S_{10.0.0.1,2,1}^{\text{SesIP}} = [I].$$

Likewise,

$$G_{10.0.0.2} = [A, C, E, G, J]$$

has gaps 10, 10, 105, 280, so it splits into

$$G_{10.0.0.2,1} = [A, C, E], \quad G_{10.0.0.2,2} = [G, J],$$

yielding

$$S_{10.0.0.2,1,1}^{\text{SesIP}} = [A, C, E], \quad S_{10.0.0.2,2,1}^{\text{SesIP}} = [G, J].$$

2-IP

Flows are grouped by unordered IP pairs. For example,

$$G_{(10.0.0.1,10.0.0.2)} = [A, C, E] \Rightarrow S_{(10.0.0.1,10.0.0.2),1}^{2\text{IP}} = [A, C, E],$$

$$G_{(10.0.0.1,10.0.0.3)} = [I] \Rightarrow S_{(10.0.0.1,10.0.0.3),1}^{2\text{IP}} = [I],$$

and

$$G_{(10.0.0.2,10.0.0.3)} = [G] \Rightarrow S_{(10.0.0.2,10.0.0.3),1}^{2\text{IP}} = [G].$$

1-Hostname (Host) Flows are grouped by logical entity identifier (here illustrated directly with hostnames). For example,

$$G_{M1} = [A, B, C, E, H, I]$$

which, with $L = 4$, yields

$$S_{M1,1}^{\text{Host}} = [A, B, C, E], \quad S_{M1,2}^{\text{Host}} = [H, I].$$

Similarly,

$$G_{M2} = [A, C, E, G, J] \Rightarrow S_{M2,1}^{\text{Host}} = [A, C, E, G], \quad S_{M2,2}^{\text{Host}} = [J].$$

1-Hostname-Session (SesH) Flows are grouped by logical entity identifier and then split into sessions when the gap between consecutive start timestamps exceeds $\Delta = 60$ s. For hostname $M1$,

$$G_{M1} = [A, B, C, E, H, I]$$

has timestamp gaps 5, 5, 10, 110, 270, so it splits into

$$G_{M1,1} = [A, B, C, E], \quad G_{M1,2} = [H], \quad G_{M1,3} = [I].$$

Thus,

$$S_{M1,1,1}^{\text{SesH}} = [A, B, C, E], \quad S_{M1,2,1}^{\text{SesH}} = [H], \quad S_{M1,3,1}^{\text{SesH}} = [I].$$

Table 2: Summary of shared neural model hyperparameters and training configuration.

Parameter	Value
Hidden dimension (d_{model})	384
Attention heads	8
Encoder layers	6
Maximum sequence length (L)	128 flows
Dropout	0.1
Optimiser	AdamW
Weight decay	0.01
LR scheduler	ReduceLROnPlateau (patience=5, factor=0.1)
Early stopping patience	12 epochs
Precision	bf16-mixed
Gradient accumulation steps	3
Max epochs	50
Max training batches/epoch	1 000
Session inactivity timeout (Δ)	300 s
Bootstrap resamples (B)	10 000

2-Hostname (2-H) Flows are grouped by unordered hostname pairs. For example,

$$G_{(M1,M2)} = [A, C, E] \Rightarrow S_{(M1,M2),1}^{2H} = [A, C, E],$$

$$G_{(M1,M3)} = [B, I] \Rightarrow S_{(M1,M3),1}^{2H} = [B, I],$$

and

$$G_{(M1,M5)} = [H] \Rightarrow S_{(M1,M5),1}^{2H} = [H].$$

5. Experimental Setup

5.1. Overview

We evaluated eight sequence grouping strategies across three Locked Shields datasets (LSPR23, LSPR24, and LSPR25), using within-year and cross-year experiments. The Original Position (OP) strategy serves as the baseline representation. In total, 485 model configurations were evaluated across sequence construction strategies, model architectures, and positional encoding settings.

5.2. Hardware and Software Environment

Experiments were executed on a Slurm-managed GPU cluster. Each training run was allocated a single NVIDIA L40S GPU (48 GB VRAM), four CPU cores (Intel Xeon Gold 6548N), and 180 GB of system memory via Slurm resource scheduling. Jobs were submitted with one GPU, one task, and four CPU cores per task. The software environment consisted of Ubuntu Linux with CUDA-enabled PyTorch and the Lightning framework for training orchestration. The total compute budget across all 485 training runs was approximately 1,608 GPU-hours on NVIDIA L40S hardware.

5.3. Model Architecture

To reduce confounding between sequence structure and model capacity, we fixed model capacity and training procedures across experiments. Neural sequence models share a common classification head, whereas encoder backbones (full attention, sparse attention, or recurrent) are treated as interchangeable components. Encoder variants are described below; Section 5.6 summarises the comparative evaluation across model families.

Longformer Encoder. For the Longformer encoder [43], we employed sliding-window sparse attention with `attention_window=128`, applied identically across all encoder layers. In the HuggingFace implementation, this corresponds to a symmetric local context of ± 64 tokens per position. Attention is bidirectional within this window. We additionally evaluated two global-attention policies: `none`, where no tokens receive global attention, and `every_16`, where global tokens are assigned at fixed positional intervals ($i \bmod 16 = 0$) for valid (non-padding) positions. Padded positions are excluded from both local and global attention via masking. No structural modifications to the canonical HuggingFace implementation were introduced.

BigBird Encoder. For BigBird [44], we used block-sparse attention with block size 16 and 3 random attention blocks per token. Although short sequences do not necessarily benefit from sparse patterns in absolute efficiency terms, we retained the block-sparse formulation to keep the BigBird backbone consistent with its intended attention mechanism and to compare sparse-attention behaviour under the same fixed sequence budget used for the other neural models. The block size was set to 16 so that a 128-flow sequence is evenly partitioned into eight blocks, while the number of random blocks was kept at the standard implementation setting of 3 [45].

BERT-style Encoder. The BERT-style encoder [46] applies full self-attention over the entire sequence without sparsity constraints, serving as a dense-attention baseline comparable to prior intrusion-detection studies.

Shared Model Configuration. To ensure a controlled comparison across grouping strategies, all neural models share the following architectural configuration:

- Hidden dimension (d_{model}): 384
- Number of attention heads (n_{head}): 8
- Number of transformer encoder layers: 6
- Maximum sequence length: 128 flows
- Longformer attention window: 128
- BigBird block size: 16
- BigBird random blocks: 3

- Dropout rate: 0.1

The shared configuration was chosen as a controlled medium-capacity operating point rather than as a separately optimised architecture for each grouping strategy. Specifically, the use of 6 layers, 8 attention heads, and dropout 0.1 follows the canonical Transformer design regime [41], while the reduced hidden size ($d_{model} = 384$) was selected to keep training tractable across dense-attention, sparse-attention, and recurrent baselines under a common hardware budget. We therefore treat these values as fixed comparison settings rather than as claims of architectural optimality. This choice is also consistent with prior findings that transformer depth and head count in NIDS are task-dependent and do not admit a single universally best configuration [12].

Flow-Level Classification and Loss. Encoders produce contextualised representations for each flow in a sequence. Classification is performed at the *flow level*: a shared linear layer is applied independently to each token embedding, yielding one logit per valid flow. Sequence padding positions are masked and excluded from both loss computation and metric evaluation. Predictions therefore correspond to individual flows within structured sequences rather than to aggregated sequence-level labels.

Training uses Binary Cross-Entropy with logits (BCEWithLogitsLoss), providing numerically stable optimisation under class imbalance.

5.4. Architectural Control Across Sequence Structures

For the neural sequence models, architectural hyperparameters were held constant across grouping strategies to isolate representational effects under fixed model capacity. In particular, the chunk size was fixed to $L = 128$ flows, so the maximum sequence length in all experiments was 128 by design. Sequences shorter than L were padded, and padding positions were excluded from model computation and evaluation where applicable. Mean realised sequence lengths still differed across grouping strategies because grouping density and session boundaries varied by representation. The choice of AdamW as optimiser is likewise a controlled design decision rather than an oversight. Because the study aims to isolate representational effects, varying the optimiser across experiments would introduce an additional confounding factor. AdamW with decoupled weight decay remains the standard optimiser for Transformer training [41] and is used consistently across all recent Transformer-based NIDS studies cited in this work. A systematic sensitivity analysis across alternative optimisers, chunk lengths, and architectural scales is left for future work.

5.5. Flow Grouping Strategies

All eight sequence grouping strategies defined formally in Section 4 were instantiated exactly as specified, without modification or additional heuristic tuning. This includes the chunking convention, canonicalisation procedures, and inactivity timeout parameter Δ for session-based strategies.

5.6. Encoder Variants and Baseline Models

To ensure that observed trends are not specific to a single modelling paradigm, we evaluated three Transformer encoder backbones and two non-attention baselines. Transformer experiments included (i) a dense full self-attention encoder (BERT-style), (ii) BigBird block-sparse attention, and (iii) Longformer sliding-window attention. Positional encoding schemes (None, Sinusoidal, and RoPE) were evaluated where compatible with the encoder implementation. A GRU-based sequential model [47] served as a recurrent baseline using identical input representations. Additionally, an XGBoost (XGB) GPU-based tabular baseline [48] was evaluated as a non-sequential reference model. In this setting, flows are treated independently without sequence modelling; *Original Position* is used only as the corresponding baseline representation.

XGBoost Tabular Baseline. The XGBoost baseline[48] was trained using GPU-accelerated histogram-based gradient boosting with learning rate = 0.05, maximum tree depth = 8, number of estimators = 800, subsample = 0.8, column subsample per tree = 0.8, regularization parameter $\lambda = 1.0$, minimum child weight = 1.0, and maximum histogram bins = 256. No explicit cap was imposed on the number of training or validation rows (`xgb_max_train_rows=0`, `xgb_max_val_rows=0`). All model families were evaluated under identical dataset splits and evaluation procedures to ensure comparability.

5.7. Positional Embedding Variations

To assess the impact of positional context encoding, we evaluate three representative schemes. *None* mirrors the configuration used in RTIDS [49], which argued that PEs are unnecessary for flow-based IDS. *Sinusoidal* is the conventional fixed scheme from the original Transformer [41]. *RoPE* [42] represents a modern relative variant. This spectrum, ranging from omitting PEs through the classic absolute encoding to a state-of-the-art relative method, allows us to isolate when positional information contributes.

For the RQ3 interaction analysis, positional-encoding comparisons are performed for all Transformer configurations with at least two comparable positional-encoding variants. This yields two-way comparisons for the BERT-style and BigBird encoders (*None* vs. *Sinusoidal*) and three-way comparisons for Longformer (*None*, *Sinusoidal*, and *RoPE*). Accordingly, RQ3 should be interpreted as a comparison of available positional-encoding choices within each encoder family rather than as a perfectly symmetric three-way positional-encoding sweep across all Transformer backbones.

5.8. Dataset Preparation and Splitting

The dataset was partitioned into training and validation sets following a standardised preprocessing pipeline described in [23].

Train and validation splits were performed at the *sequence level* based on unique grouping keys (`conn_key`), ensuring that no sequence—and therefore no flow belonging to the same grouping key—appears in both splits. To mitigate

extreme class imbalance in validation data, we applied a controlled stratified sampling procedure at the sequence level with a fixed random seed (42). This guarantees deterministic reproducibility and prevents cross-split information leakage. Flows were converted into tensor representations containing normalised features including ports, protocols, byte and packet counts, and timing statistics. The complete list of the 83 used input features is provided in Appendix A. The processed sequences were stored in Parquet files to facilitate efficient data loading during model training. We employed a custom Lightning DataModule implementation to load and batch the data. Each batch contained sequences padded to a uniform length, along with associated masks indicating padded positions. Due to the computational intensity of training on large sequences, a batch size tuning routine was implemented to maximise GPU utilisation without encountering memory constraints.

5.9. Optimisation and Training Protocol

Neural sequence models (Transformer and GRU) were optimised using AdamW with decoupled weight decay. Bias parameters and LayerNorm weights were excluded from weight decay (0.0), while all remaining parameters used weight decay 0.01. The initial learning rate was determined using a learning-rate finder and then applied uniformly during training. A ReduceLROnPlateau scheduler monitored validation loss (`val_loss`) in `mode=min`, with reduction factor 0.1, `patience=5` epochs, and minimum learning rate 10^{-7} . Early stopping monitored the primary evaluation metric (`val_avgprecision`) in `mode=max`, with `patience=12` and minimum improvement threshold $\Delta = 10^{-3}$. Each experiment was trained on a single GPU using mixed precision (`bf16-mixed`) with gradient accumulation (`accumulate_grad_batches=3`). Prior to training, the per-experiment batch size was selected using Lightning’s batch-size scaling (binary search) on a synthetic workload; for GRU models, the resulting batch size was additionally scaled by a safety factor to avoid out-of-memory failures. Model training was performed with a minimum of 10 epochs and a maximum of 50 epochs. Each epoch was limited to at most 1000 training batches (`limit_train_batches=1000`). A hard cap on optimiser steps was enforced (`max_steps=5000` for transformer models and `max_steps=20000` for GRU models).

Optimisation procedures were intentionally held constant across grouping strategies to reduce confounding when comparing representational effects. For the XGBoost baseline, training was performed independently using GPU-accelerated gradient boosting with identical dataset splits; the full XGBoost hyperparameter configuration is reported in Section 5.6.

5.10. Evaluation Metrics

Performance across experiments was evaluated using both threshold-independent ranking metrics and threshold-dependent classification metrics. Because intrusion detection is characterised by strong class imbalance, we treat Average Precision (AP) as the primary metric. Precision–recall analysis is more informative

than ROC analysis under severe imbalance because precision directly reflects the false-alarm burden among predicted positives[50, 51]. By contrast, threshold-dependent metrics such as F_1 remain useful as secondary operational indicators, but may vary substantially with the selected decision threshold.

- **Average Precision (AP):** Primary ranking metric, equivalent to the area under the Precision–Recall curve. AP is particularly informative under class imbalance because it focuses on the ranking quality of positive samples.
- **AP Lift:** To account for extreme class imbalance, we report *Average Precision Lift*, defined as

$$AP_{Lift} = \frac{AP}{\pi}$$

where π denotes the positive class prevalence in the evaluation dataset. AP Lift therefore measures how many times better the model ranks malicious events compared to random guessing. A value of 1 corresponds to random performance, while higher values indicate stronger detection capability. AP Lift provides a prevalence-normalised interpretation of detection performance, enabling fair comparison across datasets with different attack ratios.

- F_1 : The harmonic mean of precision and recall computed at a fixed decision threshold. This metric provides an operational perspective on detection performance.
- **Area Under the ROC Curve (AUROC):** Threshold-independent measure of separability between benign and malicious traffic.
- **Accuracy, Precision, and Recall:** Additional threshold-dependent metrics included for completeness.
- **Computational Efficiency:** Training time and prediction throughput were recorded to assess the practical deployment cost of different sequence constructions. For comparative efficiency analysis in RQ4, we use *AP per million processed tokens* as the primary normalised efficiency metric. This metric relates ranking quality to prediction-time token processing cost and is intended to compare configurations under differing induced sequence workloads. It should therefore be interpreted as a normalised prediction-efficiency measure rather than as a complete substitute for end-to-end latency or infrastructure-specific deployment benchmarks.

5.11. Statistical Evaluation and Confidence Intervals

To estimate uncertainty in the primary ranking metric, we report non-parametric bootstrap confidence intervals for Average Precision (AP) using

$B = 10,000$ resamples. Resampling was performed at the *sequence level* to preserve the temporal and structural dependencies induced by the grouping strategies, rather than resampling individual flows i.i.d. For each bootstrap replicate, entire sequences were sampled with replacement, and all flows contained within the sampled sequences were included in the evaluation set. Although predictions are generated at the flow level, performance metrics (AP, AUROC, F_1 , Precision, Recall, Accuracy) were computed over the aggregated set of flows in each bootstrap sample. Reported confidence intervals correspond to the 2.5th and 97.5th percentiles of the resulting empirical distributions. This design follows the general principle that uncertainty estimation should respect the dependence structure induced by the representation and evaluation pipeline [52, 53]. The reported bootstrap intervals therefore quantify evaluation uncertainty conditional on the chosen train/validation split, but do not capture split-to-split variability. This single-split design is a limitation of the present study: with 485 runs across 8 strategies, 3 datasets, and multiple model families, repeating all experiments across multiple random seeds was computationally prohibitive within the available budget of 1,608 GPU-hours. The bootstrap intervals partially mitigate this concern by quantifying evaluation-side uncertainty, and we note that the key comparative conclusions (e.g., the 2-Hostnames advantage on LSPR25) involve AP differences of 0.09–0.16, which substantially exceed the observed bootstrap CI widths.

5.12. Reproducibility

We released the LSPR25 dataset publicly via Zenodo [22], extending the previously released LSPR23 and LSPR24 datasets [23, 24]. In addition, all reported experiments were tracked in MLflow, including configuration parameters, model checkpoints, training and validation metrics, and run metadata. The archived experiment artefacts support traceability of the reported results and enable re-execution of the evaluation pipeline. The experiment code is available at <https://gitlab.nlta.tech/allard.dijk/flowseq>. Where redistribution is constrained by infrastructure-specific dependencies, we report the exact methodological details, hyperparameter settings, grouping definitions, and feature lists required for independent replication.

6. Experimental Results

This section evaluates the impact of sequence construction on flow-based intrusion detection using the Locked Shields datasets. Performance is ranked by Average Precision (AP) within each evaluation dataset; threshold-dependent metrics such as F_1 are reported only as supporting indicators. Because class prevalence differs substantially between datasets, we do not report a single global ranking across years.

The analysis is organised around the four research questions defined in Section 1. First, we evaluate how sequence construction influences detection performance under class imbalance (RQ1). Second, we analyse robustness under

Table 3: Family-aware RQ1 comparison by evaluation dataset. For each dataset, the table reports the best-performing XGBoost, GRU, and Transformer configuration ranked by Average Precision (AP). XGBoost operates on individual flow features without sequence context; its results are included as a non-sequential baseline. The final column reports the AP difference between the best Transformer and the best XGBoost configuration within the same dataset.

Eval. Dataset	XGB AP	Best GRU Sequence	GRU AP	Best Transf. Sequence	Transf. Encoder	Transf. AP	Δ AP (Tr.-XGB)
LSPR23	1.0000	SesH	0.9984	2-H	longformer	0.9964	-0.0036
LSPR24	0.9923	2-IP	0.7577	2-H	bigbird	0.9897	-0.0026
LSPR25	0.8513	SesH	0.5614	2-H	longformer	0.9411	+0.0898

cross-year distribution shift using fair-selection generalisation from same-year to next-year data (RQ2). Third, we investigate the interaction between positional encodings and the structural properties of constructed sequences (RQ3). Finally, we examine the trade-off between detection performance and computational efficiency using Pareto-style analysis of AP versus inference cost (RQ4).

For RQ3 and RQ4, comparisons are likewise interpreted within the relevant evaluation context: positional encoding effects are analysed as configuration-dependent interactions, and efficiency leaders are reported per evaluation dataset rather than as a single global ranking.

For clarity, each research question is summarised using one compact table and one representative figure.

6.1. RQ1: Impact of sequence construction under class imbalance

We first investigate how different sequence construction strategies influence detection performance under varying levels of class imbalance. To make this comparison informative across model families, we report the strongest XGBoost, GRU, and Transformer configuration within each evaluation dataset and interpret them side by side. This allows RQ1 to address not only which configuration performs best overall, but also whether more complex sequence-aware deep learning models provide a meaningful advantage over a strong shallow baseline. Taken together, the results show that sequence construction becomes increasingly decisive as detection difficulty grows: the most effective representation depends on dataset characteristics, class imbalance, and its interaction with the model family.

Table 3 provides a model family-focused comparison for each evaluation dataset. Two observations stand out. First, on LSPR23 and LSPR24, XGBoost remains marginally ahead of the best Transformer, with AP differences of only -0.0036 and -0.0026, respectively. This indicates that on these two datasets, detection performance is near-ceiling across model families: XGBoost is ranked first, but the best Transformer is already very close, leaving limited headroom for any architecture to demonstrate additional benefit. In contrast, the GRU baseline trails more clearly behind on both datasets, suggesting that recurrent sequence modelling does not recover the same level of discriminative power as either the tabular XGBoost baseline or the strongest Transformer variants.

Second, the pattern changes on LSPR25. Here, the best Transformer configuration, using the **2-Hostnames** grouping strategy with a Longformer encoder, achieves an AP of 0.9411, exceeding the best XGBoost result by 0.0898. This is the clearest evidence in RQ1 that sequence construction interacts with model family in a non-trivial way. Under the more challenging and highly imbalanced LSPR25 setting, a structured sequence representation combined with a sequence-aware Transformer becomes materially more effective than the XGBoost, which processes individual flows without sequence context. At the same time, the GRU baseline remains substantially lower (0.5614 AP), indicating that the gain is not a generic property of deep learning, but is specific to certain architectures and representations.

These results support a more nuanced interpretation of RQ1 than a simple single-winner summary. For the LSPR23 and LSPR24 datasets, where the class imbalance is less severe than in LSPR25 (positive-class prevalence of approximately 30.0% and 7.1% under OP, versus 2.2% for LSPR25; see Table 8 for per-strategy values), detection performance is near-ceiling for all model families, leaving limited headroom for structured representations to demonstrate additional benefit. On the most demanding year, however, sequence construction becomes the decisive design factor: the best-performing model shifts to a Transformer with 2-Hostname grouping, and the +0.090 AP advantage over XGBoost is the largest inter-family gap observed in the study. This conditional pattern is itself informative: sequence-aware models deliver their greatest advantage precisely in the conditions where accurate detection is hardest to achieve.

Figure 1 complements Table 3 by showing the full configuration space in terms of AP and F_1 . While both metrics are closely aligned for LSPR23 and, to a lesser extent, LSPR24, their behaviour diverges more significantly under severe class imbalance. In LSPR25, several configurations retain relatively high AP despite much lower F_1 , indicating that threshold-dependent metrics can understate ranking quality when positive instances are extremely rare. This reinforces the use of AP as the primary metric for RQ1 and helps explain why the Transformer advantage on LSPR25 is more clearly expressed in ranking quality than in any single thresholded operating point. From an operational triage perspective, precision@1 further confirms the difficulty gradient: on LSPR23, 33 configurations achieve near-perfect triage precision, whereas on LSPR25 only the XGBoost baseline reaches this level, underscoring the value of the structured representations that achieve high AP under these conditions.

◆ Restricting the comparison to sequence-aware models, 2-Hostnames emerges as the most effective grouping strategy for Transformer encoders across all three evaluation years. On LSPR23, the advantage over the next-best Transformer representation (1-Hostname) is narrow (AP difference: +0.0060), suggesting that most discriminative structure is already accessible through global chronology alone. On LSPR24, however, the gap widens noticeably (+0.0669 AP over the best chronologically ordered Transformer), and on LSPR25 it becomes decisive: 2-Hostnames achieves 0.9411 AP, exceeding the next-best Transformer strategy (SesH, 0.7828 AP) by +0.1583. This progressive widening of the 2-Hostnames advantage as dataset complexity increases indicates that hostname-

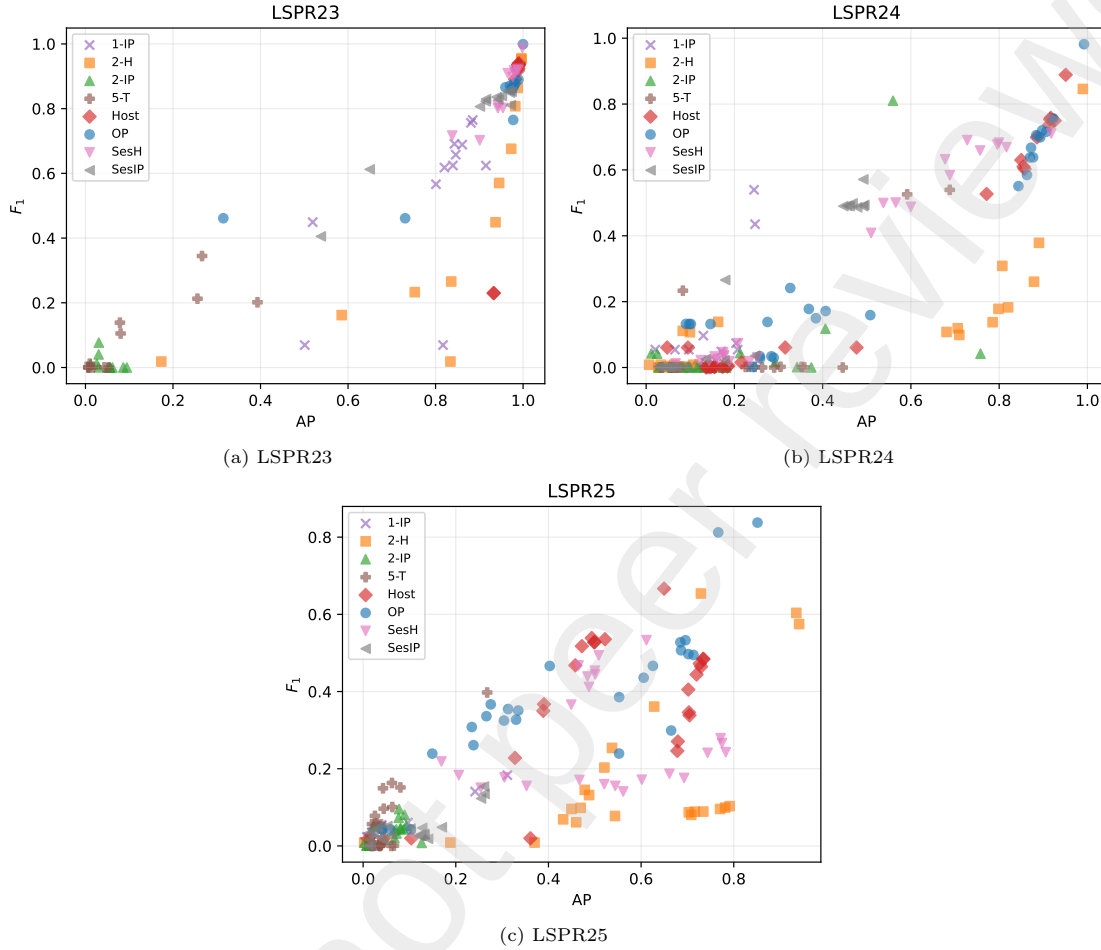


Figure 1: Average Precision (AP) versus F_1 across datasets. Each point represents a model configuration evaluated on the corresponding evaluation dataset.

pair grouping captures contextual structure that becomes increasingly beneficial under severe class imbalance. For the GRU family, no single grouping strategy dominates consistently, although SesH shows promise by ranking first among GRU configurations on both LSPR23 and LSPR25. These results reinforce that sequence construction is a consequential design factor for sequence-aware models, with 2-Hostnames providing the strongest and most consistent benefit for Transformer-based detection.

6.2. RQ2: Robustness under cross-year distribution shift

To assess robustness under temporal distribution shift, we evaluate how configurations selected on one dataset generalise to subsequent years of the Locked Shields data. Specifically, we perform a *fair-selection generalisation* analysis:

Table 4: Worst fair-selection drift cases (selected on same-year, evaluated on the next year). All listed cases are from the LSPR23→LSPR24 transition, which exhibits the largest absolute AP drops; the worst LSPR24→LSPR25 case ($\Delta\text{AP} = -0.748$) falls below this threshold.

Transition	Seq.	Model / Configuration	AP same	AP next	ΔAP	Lift same	Lift next
LSPR23→24	2-H	transformer (longf./Rope/every_16)	0.988	0.027	-0.961	106.16	6.33
LSPR23→24	2-H	transformer (longf./Rope/none)	0.982	0.034	-0.948	105.56	7.87
LSPR23→24	SesIP	transformer (bert/Sinusoidal)	0.972	0.027	-0.945	23.02	1.00
LSPR23→24	2-H	transformer (longf./Sin./every_16)	0.996	0.056	-0.940	107.05	13.00
LSPR23→24	SesIP	transformer (longf./Rope/none)	0.970	0.031	-0.938	22.97	1.15
LSPR23→24	SesIP	transformer (longf./Rope/every_16)	0.967	0.029	-0.938	22.90	1.05
LSPR23→24	SesIP	gru (bi=0)	0.974	0.042	-0.931	23.07	1.56
LSPR23→24	2-H	transformer (longf./Sin./none)	0.996	0.074	-0.922	107.09	17.25
LSPR23→24	2-H	transformer (longf./None/every_16)	0.973	0.057	-0.916	104.57	13.18
LSPR23→24	SesIP	transformer (longf./None/none)	0.952	0.052	-0.899	22.54	1.92

Table 5: Model family-focused same-year versus next-year comparison by transition. For each train-to-next-year transition, the table reports, within each model family, the configuration selected by same-year performance together with its same-year Average Precision (AP), next-year AP, and the resulting generalisation gap (next-year minus same-year). XGBoost does not use sequence construction.

Transition	XGB			Seq.	GRU			Transformer				
	AP same	AP next	Gap		AP same	AP next	Gap	Seq.	Enc.	AP same	AP next	Gap
LSPR23→24	1.0000	0.2416	-0.7584	SesH	0.9984	0.1758	-0.8226	2-H	longf.	0.9964	0.0742	-0.9222
LSPR24→25	0.9923	0.7666	-0.2257	2-IP	0.7577	0.0097	-0.7480	2-H	bigbird	0.9897	0.7291	-0.2607

configurations are selected based on their performance on a same-year dataset and subsequently evaluated on the next available year. This setup reflects a realistic operational scenario in which models are tuned on historical traffic but deployed on future network environments.

Table 4 highlights representative cases in which configurations selected based on same-year performance exhibit substantial degradation when evaluated on the next year. In several cases, AP drops by more than 0.9, showing that configurations that appear near-perfect under same-year evaluation can collapse under future-year deployment. These failures occur across multiple sequence construction strategies and model types, indicating that the degradation is not confined to a single architecture but instead reflects broader distributional differences between years.

Table 5 provides a complementary model family-focused view of this effect. For each train-to-next-year transition, it reports, within the XGBoost, GRU, and Transformer families, the configuration selected by same-year performance

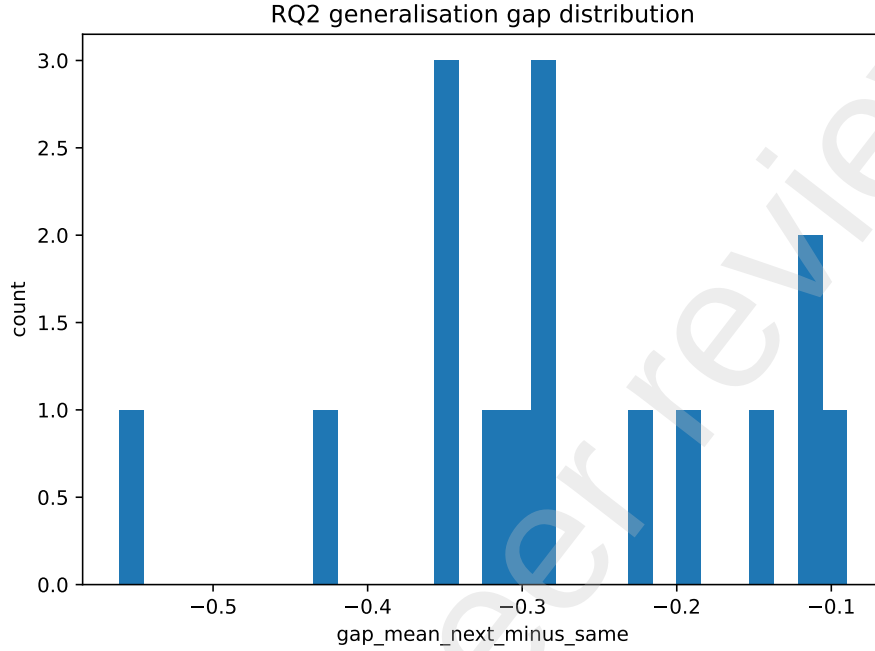


Figure 2: Distribution of generalisation gaps under cross-year evaluation, measured as next-year performance minus same-year performance. Negative values indicate performance degradation when moving from the evaluation year used for selection to a subsequent year.

together with its same-year AP, next-year AP, and resulting generalisation gap. This view makes two points clear. First, substantial degradation is not limited to isolated runs, but is observed across all three model families. Second, the magnitude of degradation is family-dependent. For LSPR23→LSPR24, the XGBoost baseline remains the least affected family, whereas the selected Transformer configuration exhibits the largest AP drop. For LSPR24→LSPR25, XGBoost again shows the smallest degradation, while the selected Transformer remains relatively competitive and the selected GRU configuration deteriorates much more severely. Thus, robustness under cross-year shift depends not only on sequence construction, but also on the interaction between representation and model family.

Figure 2 shows that generalisation gaps are predominantly negative across evaluated configurations, confirming that performance usually degrades under cross-year distribution shift. This finding indicates that strong same-year results do not reliably transfer to future Locked Shields datasets. In other words, model selection based solely on same-year evaluation leads to overly optimistic estimates of operational performance.

In the fair-selection analysis, 178 of 194 next-year comparisons (91.75%) show a decrease in AP. These 194 comparisons correspond to 97 evaluated config-

urations assessed over the two adjacent cross-year transitions, LSPR23→LSPR24 and LSPR24→LSPR25. Notably, the two transitions exhibit qualitatively different failure modes. LSPR23→LSPR24 is more severe on average (median $\Delta AP = -0.731$), although 16.5% of configurations show a nominal improvement. However, all 16 of these improving configurations start from very low same-year AP (mean 0.033) and do not reach competitive next-year performance; they are weak configurations that happen to score marginally better on the next year rather than genuinely robust models. LSPR24→LSPR25 is milder in median terms ($\Delta AP = -0.298$) but 100% of configurations degrade without exception. Across both transitions, same-year AP is strongly anti-correlated with the cross-year gap (Pearson $r = -0.87$), indicating that configurations achieving the highest within-year performance tend to suffer the largest absolute degradation under temporal shift. Negative gaps are observed across all evaluated model families and every grouping strategy. Moreover, the median next-year AP gap is negative for all grouping strategies, ranging from -0.0675 for 2-IP to -0.5904 for OP, indicating that no evaluated representation is consistently robust across years.

Overall, these results demonstrate that configurations that appear highly effective within a single year remain strongly sensitive to temporal drift. Same-year model selection therefore does not reliably predict next-year performance. Robust evaluation requires cross-year validation to determine whether learned traffic patterns generalise beyond the dataset used for training and tuning.

6.3. RQ3: Interaction between positional encodings and sequence structure

We use the term *sequence structure* to refer to the degree to which a grouping strategy preserves coherent behavioural locality and temporally contiguous entity-level context within a sequence, as formalised in Section 4.1. In addition to sequence construction itself, transformer-based architectures rely on positional encodings to represent the order of elements within a sequence. We therefore evaluate how different positional encoding strategies interact with the grouping strategies introduced in this study. We include this analysis not as a separate contribution but as a necessary control: if positional encoding effects were to dominate or confound the grouping-strategy comparison, the conclusions of RQ1 and RQ2 would be weakened. The analysis below demonstrates that PE effects are secondary to sequence construction, thereby validating the interpretation that grouping strategy—rather than positional encoding—is the primary representational factor.

To avoid ambiguity, we distinguish between two quantities. *PE sensitivity* denotes the AP range across all evaluated positional-encoding variants for the same configuration, i.e.,

$$\max_{PE} AP - \min_{PE} AP.$$

By contrast, *best-vs-baseline gain* denotes the AP improvement of the best positional encoding relative to a designated baseline encoding for that configuration. A configuration may therefore exhibit high sensitivity even when the baseline

Table 6: Top scenarios where positional embedding choice most affects AP (highest sensitivity).

Train / Eval.	Seq.	Encoder / Policy	Base-line PE	Best PE	Δ AP	Sensitivity
24 / 24	2-IP	bigbird	None	None	0.000	0.377
23 / 23	2-H	bert	None	Sin.	0.352	0.352
23 / 23	1-IP	bert	None	Sin.	0.327	0.327
24 / 25	Host	bert	None	None	0.000	0.322
24 / 25	SesH	longf. / every_16	None	None	0.000	0.302
23 / 23	SesIP	bigbird	None	None	0.000	0.290
25 / 25	1-IP	longf. / every_16	None	None	0.000	0.287
25 / 25	SesH	bigbird	None	Sin.	0.277	0.277
24 / 24	2-IP	longf. / every_16	None	None	0.000	0.259
24 / 24	2-IP	bert	None	None	0.000	0.258

encoding is already the best-performing choice and the best-vs-baseline gain is zero or negative.

Table 6 summarises configurations with the highest sensitivity to positional encoding choice, where sensitivity is defined as the AP range between the best and worst positional-encoding variant evaluated for the same configuration. The most sensitive cases now occur across multiple encoder backbones rather than within a single Transformer variant: among the highest-sensitivity scenarios, BERT, BigBird, and Longformer all appear. In several cases, positional encoding materially affects ranking quality, with AP ranges exceeding 0.25 and reaching 0.377 in the most sensitive configuration. However, sensitivity should not be conflated with guaranteed improvement over a baseline encoding: in some configurations the baseline already attains the best AP, whereas in others an alternative encoding yields a substantial gain. No single positional encoding emerges as universally best; depending on the configuration, the strongest results are obtained with **None**, **Sinusoidal**, or **RoPE**.

Figure 3 illustrates the relationship between positional encoding sensitivity and achievable detection performance. The results show that positional encoding effects are highly configuration-dependent. In some cases, the best-performing positional encoding differs substantially from the baseline and leads to large AP gains, whereas in other cases the baseline encoding already performs best. This indicates that positional encoding should not be treated as a universally beneficial addition, but rather as a design choice whose value depends on the interaction between dataset, grouping strategy, encoder, and attention policy. This pattern is more pronounced in same-year than in cross-year evaluation: median PE sensitivity is 0.0773 for same-year scenarios versus 0.0317 for cross-year scenarios, and the corresponding median best-vs-baseline gains are 0.0195 and 0.0101.

Across the 160 evaluated positional-encoding scenarios, the median sensitivity is modest (0.0475), and the median best-vs-baseline gain is 0.0081. However,

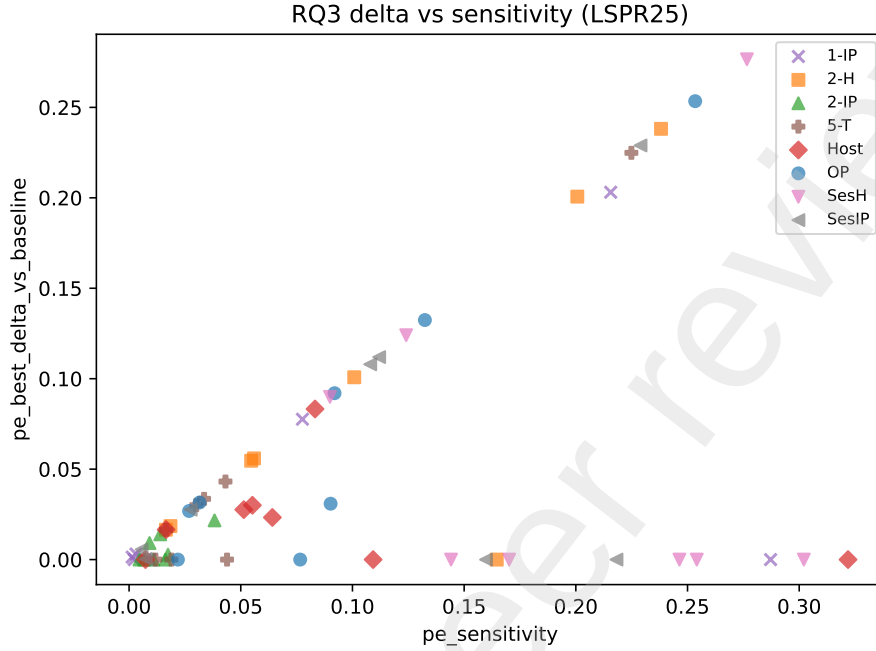


Figure 3: Relationship between positional-encoding sensitivity and best-vs-baseline improvement on LSPR25. Each point represents a configuration evaluated with multiple positional-encoding variants. The horizontal axis shows PE sensitivity, defined as the AP range across evaluated positional encodings for the same configuration, while the vertical axis shows the AP gain of the best positional encoding relative to the designated baseline encoding.

the upper tail is substantial: PE sensitivity exceeds 0.2446 in the top decile of cases and reaches 0.3771 in the most sensitive configuration, while the best-vs-baseline gain reaches 0.3515 in the strongest improvement case. These effects are also representation-dependent. Highly sensitive cases occur across BERT, Big-Bird, and Longformer, and the strongest gains are concentrated in only a subset of sequence constructions rather than appearing uniformly across all representations. Overall, these observations indicate that the effectiveness of positional encoding is conditional rather than universal. Positional encoding should therefore be treated as a joint design choice with sequence construction and encoder selection, rather than as a universally beneficial component.

To assess whether clearer patterns emerge among the most performant configurations, we additionally restrict the analysis to same-year evaluations achieving $AP > 0.95$, yielding 19 configurations. For these highly performant configurations, the None baseline is optimal in 9 of 19 cases, Sinusoidal in 8, and RoPE in only 2. The median PE sensitivity is 0.023 and the maximum is 0.179, confirming that positional encoding effects remain modest even when filtering for the strongest configurations. Among the 19 top-performing cases, Longformer

Table 7: Most compute-efficient configurations per dataset (measured by AP per million tokens).

Dataset	Seq.	Model	Encoder / PE / Policy	AP	AP/M tokens
LSPR23	OP	xgb	—	1.000	0.326
LSPR24	OP	xgb	—	0.992	0.246
LSPR25	2-H	transformer	longf. / Sin. / every_16	0.941	0.127

accounts for 10 configurations, suggesting that the sparse-attention mechanism interacts favourably with structured sequence representations regardless of PE choice. These filtered results reinforce the overall finding: PE selection rarely determines whether a configuration reaches a high performance, but Sinusoidal encoding provides a slight edge when a PE variant does yield measurable improvement.

6.4. RQ4: Performance-efficiency trade-off

While the previous research questions focus on detection performance, practical intrusion detection systems must also consider computational efficiency. In large-scale network monitoring environments, models must process substantial volumes of network flows, making inference throughput and computational cost important design factors. We therefore analyse the trade-off between detection performance and computational efficiency across sequence construction strategies and model architectures.

We use AP per million processed tokens as a normalised proxy for inference efficiency across configurations, since sequence construction directly affects token volume and thus the effective workload presented to the model. This metric is not intended as a universal measure of systems cost, but as a comparable workload-normalised efficiency measure within the present evaluation setting.

Table 7 reports the most compute-efficient configuration per dataset, measured using the AP-per-million-tokens metric introduced in Section 5.

For LSPR23 and LSPR24, the most efficient configuration is XGBoost, which combines near-perfect detection performance with no sequence construction overhead. By contrast, LSPR25 exhibits a different efficiency landscape: the efficiency leader shifts to the same configuration that also achieves the highest AP overall, namely a transformer model using **2-Hostnames** grouping, a Longformer encoder, **Sinusoidal** positional encoding, and periodic global attention (**every_16**).

Table 8 provides structural context for these efficiency results. Sequence construction changes not only model input organisation but also the shape of the workload itself, including the number of induced training sequences, sequence fragmentation, positive-class prevalence (π), and effective coverage per epoch under the fixed batch budget. Strategies such as **5-T**, **SesH**, and **SesIP** generate very large numbers of short sequences, which reduces effective coverage and increases batching overhead. By contrast, OP and several pair-based strategies often produce longer sequences and fewer training examples, which can improve

Table 8: Sequence dataset overview by evaluation dataset and sequence construction strategy. π : positive class ratio. Train seq.: number of training sequences. Length statistics (p50/p90/p99): percentiles of the sequence-length distribution. Coverage/epoch: effective percentage of training data processed per epoch under the fixed batch limit. Passes/epoch: corresponding number of dataset passes.

Data-set	Seq.	π	Train seq.	p50	p90	p99	Cov./epoch (%)	Passes/epoch
LSPR23	OP	0.2998	102 209	128	128	128	490.2	4.90
LSPR23	2-H	0.0093	215 253	11	128	128	413.5	4.13
LSPR23	2-IP	0.0054	217 452	5	128	128	409.3	4.09
LSPR23	Host	0.1299	227 684	128	128	128	312.7	3.13
LSPR23	1-IP	0.0359	227 864	128	128	128	219.9	2.20
LSPR23	5-T	0.0018	4 548 581	1	5	30	170.9	1.71
LSPR23	SesH	0.1270	16 306 410	1	2	15	47.7	0.48
LSPR23	SesIP	0.0422	16 707 548	1	2	15	46.5	0.47
LSPR24	OP	0.0707	126 421	128	128	128	657.3	6.57
LSPR24	2-H	0.0043	127 402	128	128	128	393.2	3.93
LSPR24	2-IP	0.0213	160 660	128	128	128	2807.2	28.07
LSPR24	Host	0.0313	253 024	128	128	128	603.9	6.04
LSPR24	1-IP	0.0279	262 103	128	128	128	317.1	3.17
LSPR24	5-T	0.0342	7 460 697	1	3	21	60.5	0.60
LSPR24	SesH	0.0317	20 081 695	1	3	8	7.6	0.08
LSPR24	SesIP	0.0273	22 682 685	1	2	7	3.9	0.04
LSPR25	OP	0.0219	230 714	128	128	128	662.3	6.62
LSPR25	2-H	0.0044	231 738	128	128	128	307.2	3.07
LSPR25	2-IP	0.0040	310 710	128	128	128	1432.5	14.33
LSPR25	Host	0.0100	461 598	128	128	128	154.2	1.54
LSPR25	1-IP	0.0049	480 264	128	128	128	148.3	1.48
LSPR25	5-T	0.0073	7 893 812	1	5	114	10.5	0.11
LSPR25	SesH	0.0111	34 952 825	1	3	8	4.4	0.04
LSPR25	SesIP	0.0053	37 373 382	1	3	7	2.4	0.02

overall efficiency even before model-side computation is considered. This efficiency gap is partly explained by GPU utilisation: session-based strategies achieve a median of 50.3% GPU utilisation versus 81.7% for other strategies, reflecting the batching overhead caused by their large number of very short sequences.

Figure 4 visualises the Pareto frontier between detection performance and computational efficiency across datasets. Each subplot corresponds to one evaluation dataset and shows the trade-off between Average Precision and the AP-per-million-tokens efficiency metric. For LSPR23 and LSPR24, XGBoost baselines dominate the frontier because they combine near-perfect AP with very low computational cost. By contrast, the LSPR25 frontier shifts toward structured sequence models, with **2-Hostnames** combined with a Longformer encoder providing the most favourable trade-off between detection performance and efficiency. These results show that the efficiency frontier is itself dataset-dependent rather than universal.

Overall, these results indicate that sequence construction influences not only detection performance but also computational efficiency. However, the

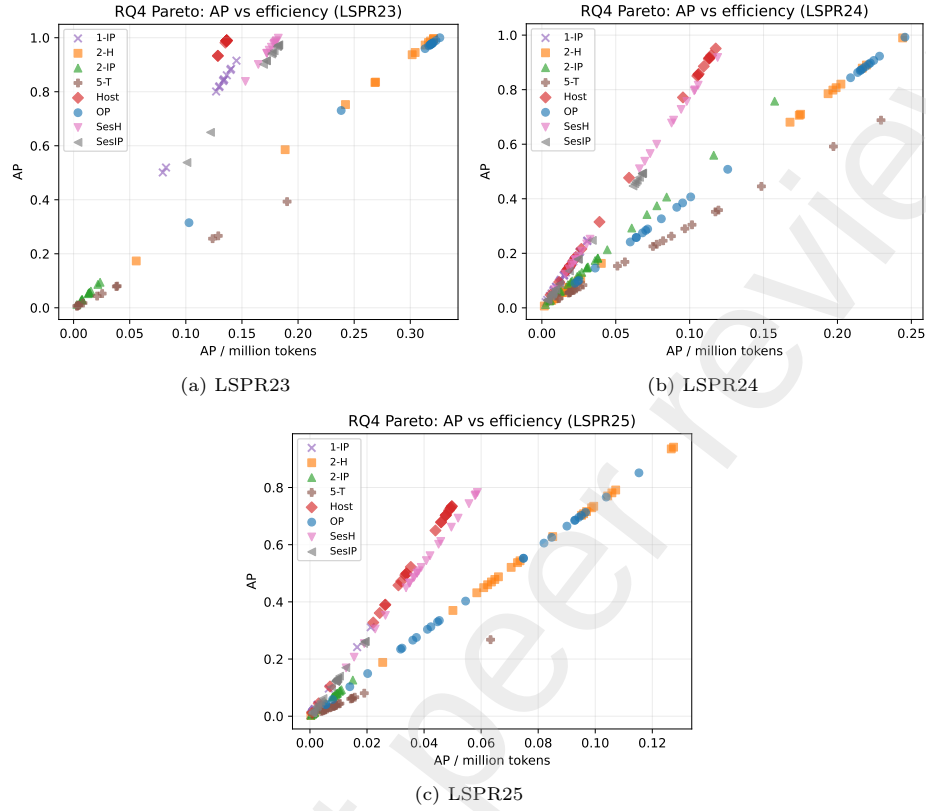


Figure 4: Pareto frontier of detection performance (Average Precision, AP) versus computational efficiency across datasets.

most favourable performance–efficiency trade-off is dataset-dependent rather than universal. For LSPR23 and LSPR24, XGBoost remains the most efficient choice, whereas for LSPR25 the efficiency leader shifts to a Longformer model using 2-Hostnames grouping. This demonstrates that efficiency is shaped not only by model family, but also by the workload structure induced by sequence construction.

7. Discussion

The results establish sequence construction as a first-class design variable in flow-based intrusion detection. Across the four research questions, the choice of grouping strategy materially determines detection quality, robustness, positional-encoding behaviour, and computational efficiency. The most effective representation depends on both dataset characteristics and model family, which means that representational choices interact strongly with the detection context. This is important because much prior work on sequence-aware intrusion detection

emphasises architectural innovation under a fixed preprocessing pipeline. Our findings indicate that part of the apparent benefit of a model may in fact depend on the structure of the sequences it is given, and that evaluating architectures without explicitly considering sequence construction risks conflating modelling gains with representational assumptions.

A second implication concerns the practical value of deep sequence models relative to strong shallow baselines. For the less challenging datasets (LSPR23 and LSPR24), the flow features already contain sufficient discriminative structure for tabular learning, so sequence-aware models offer little consistent benefit over XGBoost. On LSPR25, however, the best-performing configuration shifts to a Longformer model using 2-Hostname grouping. This gain is not explained by a more favourable induced class ratio alone: the winning representation has a lower positive class prevalence than the original dataset without sequence construction, yet achieves higher AP when paired with a Transformer. Induced positive-class prevalence partially mediates this effect: strategies that dilute the malicious signal through fine-grained grouping (e.g., 5-Tuple $\pi \approx 0.002$) perform consistently worse than those that preserve higher prevalence ($r = 0.42$ between π and best AP), reinforcing that sequence construction shapes not only contextual structure but also the effective class balance presented to the learner. Structured sequence models therefore deliver their strongest advantage under the most operationally demanding conditions—precisely the regime where baseline approaches are insufficient. Among Transformer configurations specifically, 2-Hostnames emerges as the most effective grouping strategy on all three evaluation years, with its advantage over the next-best Transformer representation widening from +0.006 on LSPR23 to +0.067 on LSPR24 to +0.158 on LSPR25. This progressive widening indicates that hostname-pair grouping captures contextual structure that becomes increasingly beneficial as detection difficulty grows.

The cross-year results yield one of the study’s most quantitatively striking findings: a Pearson correlation of $r = -0.87$ between same-year AP and next-year degradation. In the fair-selection analysis, 178 of 194 next-year comparisons show AP degradation, and every evaluated grouping strategy exhibits a negative median next-year AP gap. This strong anti-correlation provides actionable guidance: configurations achieving the highest within-year performance tend to suffer the largest absolute degradation under temporal shift, quantifying the risk of same-year-only evaluation.

◆ We emphasise that this study does not attempt to causally decompose cross-year degradation into constituent factors. In principle, degradation may arise from changes in attack types and tactics (e.g., new red-team objectives or tools), shifts in benign traffic distributions (e.g., altered defender behaviour or infrastructure scale), or changes in network topology across exercise editions. Disentangling these factors would require access to fine-grained per-attack-campaign labels and per-host traffic profiles, which are not available at the resolution needed for causal attribution. We therefore treat cross-year evaluation as an aggregate operational proxy for deployment under realistic temporal evolution rather than as evidence for any specific form of concept drift.

Thus, the issue is not limited to a few pathological runs; it reflects a broader failure of same-year model selection to predict future-year behaviour reliably. From an operational perspective, this is one of the most important findings of the study. If models are selected solely on same-year validation performance, deployment readiness can be substantially overestimated. The results therefore support using cross-year validation as part of any realistic IDS evaluation protocol, especially when the target environment is expected to evolve in topology, background traffic, defender behaviour, or attack tactics. At the same time, the fact that the magnitude of degradation differs across transitions and representations suggests that robustness is not random noise, but is itself shaped by representational choices. Although **2-Hostnames** yields the strongest result on LSPR25 within the present dataset series, its advantage depends on asset-identity metadata. The **2-IP** strategy provides a direct ablation: it applies the same pair-based grouping logic but uses raw IP addresses for all endpoints, removing the Expoid-based abstraction entirely. The resulting performance gap is large (e.g., AP 0.94 vs. 0.13 on LSPR25), confirming that the hostname-level abstraction carries substantial discriminative value. Crucially, however, the ρ mapping does not use flow-level malicious/benign labels; it uses infrastructure-role metadata that identifies which endpoints belong to the defended network. In an operational setting, defenders have access to their own asset inventory (e.g., via configuration management databases or network management systems), so this information is available without ground-truth labels. Where such asset-identity metadata is unavailable, **2-IP** remains the more deployment-realistic alternative. This gap is consistent with the dual-stack nature of the Locked Shields infrastructure, where hostname-based grouping consolidates traffic across parallel IPv4 and IPv6 addresses that IP-based grouping treats as separate pairs.

A more defensible operational approach would be to begin with a small set of historically promising grouping strategies and use the initial exercise phase for anomaly ranking and analyst triage rather than hard supervised decisions, collecting analyst-validated cases for subsequent threshold calibration or lightweight fine-tuning.

Finally, the positional-encoding and efficiency analyses show that the consequences of sequence construction extend beyond raw AP. Positional encoding is not a universally beneficial add-on: typical gains are modest, but the upper tail is large, and the most beneficial encoding depends on the encoder and the induced sequence representation. Likewise, computational efficiency is not simply a property of model family, but also of the workload generated by the grouping strategy, including sequence fragmentation, context length, and effective token volume. Because the per-epoch batch budget was held fixed across grouping strategies, representations that induce many short sequences may expose the model to a smaller fraction of the underlying dataset within a given training budget; the observed differences should therefore be interpreted as reflecting both representational properties and the workload/training budget induced by those representations.

Taken together, these findings suggest that sequence construction should be regarded as a unifying design layer that shapes what contextual informa-

tion becomes available, how useful positional information is, how well performance transfers across years, and what computational cost is incurred. At the same time, these conclusions should be interpreted as evidence from the present Locked Shields dataset series rather than as a claim of universal optimality across all flow-based IDS settings. In particular, the study prioritises controlled comparison within a consistent annual dataset family over per-configuration hyperparameter optimisation across heterogeneous benchmarks, and it interprets cross-year degradation conservatively as evidence of operational distribution shift rather than as direct proof of concept drift. Even with those boundaries, the results consistently indicate that representation design is central to realistic flow-based IDS evaluation.

8. Conclusion

This paper investigated sequence construction as a primary design factor in flow-based intrusion detection. Its primary contribution is a controlled empirical evaluation of grouping strategies across multiple years of Locked Shields data, model families, positional encodings, and computational-efficiency conditions; a secondary contribution is the release of the LSPR25 dataset to support future reproducible evaluation under temporal distribution shift.

The central empirical finding is that sequence construction becomes increasingly decisive as detection difficulty grows. Among Transformer configurations, 2-Hostname grouping achieves the highest AP on all three evaluation years, with its advantage widening from +0.006 on LSPR23 to +0.067 on LSPR24 to +0.158 on LSPR25. On the most challenging LSPR25 dataset, this strategy paired with a Longformer encoder achieves AP 0.9411, exceeding the XGBoost baseline by +0.090 despite a positive-class prevalence of only 0.44%. On the less demanding datasets, flow-level features are sufficient for near-ceiling performance across model families, which means that the value of structured sequence representations is context-dependent. This conditional pattern is itself informative: sequence construction matters most precisely when detection is hardest.

Cross-year evaluation revealed a strong anti-correlation ($r = -0.87$) between same-year AP and next-year degradation, with 178 of 194 fair-selection comparisons showing AP decline. This quantifies the risk of same-year-only model selection and argues for incorporating cross-year validation into realistic IDS evaluation protocols.

The study also highlights that representational choices affect more than predictive quality alone. Positional encoding is a conditional design choice rather than a universally beneficial component: typical gains are modest, but highly sensitive cases occur across multiple Transformer backbones, and the preferred encoding varies across encoder and sequence-construction settings. Likewise, the most attractive performance-efficiency trade-off is representation- and dataset-dependent. Sequence construction therefore shapes not only what contextual information a model can exploit, but also how robustly that information transfers across years and what computational cost is incurred.

Taken together, these findings support treating sequence construction as a first-class design dimension in flow-based intrusion detection rather than as a fixed implementation detail.

From an operational perspective, the cross-year degradation results suggest that prior-year models should not be treated as plug-and-play detectors for future exercises. A more defensible approach is to use prior-year results to narrow the representational design space—prioritising 2-Hostname grouping as a performance-oriented candidate and 2-IP as a robustness-oriented alternative (smallest median next-year AP degradation at -0.068 , with only 62.5% of comparisons showing degradation)—while relying on anomaly ranking and analyst triage rather than hard supervised decisions during initial deployment.

Future work will extend this analysis to additional Locked Shields datasets and more challenging attack classes, including lateral movement and other multi-stage behaviours. We also plan to investigate label-efficient and human-in-the-loop adaptation strategies that leverage pretrained sequence models with minimal analyst-validated samples for threshold calibration or lightweight fine-tuning during future exercise deployment.

CRedit authorship contribution statement

Allard Dijk: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing.

Risto Vaarandi: Formal analysis, Methodology, Supervision, Writing – review & editing.

Roland Meier: Supervision, Writing – review & editing.

Mauno Pihelgas: Data curation, Writing – review & editing.

Funding

This work was co-funded by the European Union and Estonian Research Council via project TEM-TA5.

Data availability

LSPR25 is publicly available via Zenodo [22]. The previously released LSPR23 and LSPR24 datasets are available via the sources cited in this paper [23, 24].

Declaration of generative AI in the writing process

During the preparation of this work, the authors used ChatGPT in order to improve language, structure, and readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was conducted with the support of the NATO Cooperative Cyber Defence Centre of Excellence (CCDCOE), Tallinn University of Technology, and the Netherlands Defence Academy, and co-funded by the European Union and Estonian Research Council via project TEM-TA5. We also thank Clarified Security, as well as the dedicated members of the Locked Shields organising teams (White, Red, Green, and Yellow) for making the exercise a remarkable event. The authors would like to express their gratitude to Peter Hladky, Robert van de Ketterij, Bernhard Lippe, Cosimo Melella, Tavi Pöldma, Taavi Sonets, and Dan Ungureanu. Finally, we acknowledge NRG Pallas, with special thanks to Martijn Nuijens for leading the Blue Team we based our research on.

References

- [1] J. B. Awotunde, F. E. Ayo, R. Panigrahi, A. Garg, A. K. Bhoi, P. Barsocchi, A multi-level random forest model-based intrusion detection using fuzzy inference system for internet of things networks, *International Journal of Computational Intelligence Systems* 16 (1) (2023) 31.
- [2] M. T. Abdelaziz, A. Radwan, H. Mamdouh, A. S. Saad, A. S. Abuzaid, A. A. Abdelhakeem, S. Zakzouk, K. Moussa, M. S. Darweesh, Enhancing network threat detection with random forest-based nids and permutation feature importance, *Journal of Network and Systems Management* 33 (1) (2025) 2.
- [3] S. Wali, Y. A. Farrukh, I. Khan, Explainable ai and random forest based reliable intrusion detection system, *Computers & security* 157 (2025) 104542.
- [4] R. Devendiran, A. V. Turukmane, Dugat-lstm: Deep learning based network intrusion detection system using chaotic optimization strategy, *Expert Systems with Applications* 245 (2024) 123027. doi:<https://doi.org/10.1016/j.eswa.2023.123027>. URL <https://www.sciencedirect.com/science/article/pii/S0957417423035297>
- [5] M. Gamal, M. Elhamahmy, S. Taha, H. Elmahdy, Improving intrusion detection using lstm-rnn to protect drones' networks, *Egyptian Informatics Journal* 27 (2024) 100501. doi:<https://doi.org/10.1016/j.eij.2024.100501>. URL <https://www.sciencedirect.com/science/article/pii/S1110866524000641>

- [6] A. K. B. Arnob, M. F. Mridha, M. Safran, M. Amiruzzaman, M. R. Islam, An enhanced lstm approach for detecting iot-based ddos attacks using honeypot data, *International Journal of Computational Intelligence Systems* 18 (1) (2025) 19.
- [7] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, M. Portmann, E-graphsage: A graph neural network based intrusion detection system for iot, in: *NOMS 2022-2022 IEEE/iFIP network operations and management symposium*, IEEE, 2022, pp. 1–9.
- [8] R. Xu, G. Wu, W. Wang, X. Gao, A. He, Z. Zhang, Applying self-supervised learning to network intrusion detection for network flows with graph neural network, *Computer Networks* 248 (2024) 110495.
- [9] Z. Sun, A. M. Teixeira, S. Toor, Gnn-ids: Graph neural network based intrusion detection system, in: *Proceedings of the 19th international conference on availability, reliability and security*, 2024, pp. 1–12.
- [10] L. G. Nguyen, K. Watabe, Flow-based network intrusion detection based on bert masked language model, in: *Proceedings of the 3rd International CoNEXT Student Workshop, CoNEXT-SW '22*, Association for Computing Machinery, New York, NY, USA, 2022, p. 7–8. doi:10.1145/3565477.3569152.
- [11] Z. Long, H. Yan, G. Shen, X. Zhang, H. He, L. Cheng, A transformer-based network intrusion detection approach for cloud security, *Journal of Cloud Computing* 13 (1) (2024) 5.
- [12] L. D. Manocchio, S. Layeghy, W. W. Lo, G. K. Kulatilleke, M. Sarhan, M. Portmann, Flowtransformer: A transformer framework for flow-based network intrusion detection systems, *Expert Systems with Applications* 241 (2024) 122564. doi:10.1016/j.eswa.2023.122564.
- [13] L. G. Nguyen, K. Watabe, A method for network intrusion detection using flow sequence and bert framework, in: *ICC 2023 - IEEE International Conference on Communications*, IEEE, 2023, p. 3006–3011. doi:10.1109/icc45041.2023.10279335.
- [14] X. Han, S. Cui, S. Liu, C. Zhang, B. Jiang, Z. Lu, Network intrusion detection based on n-gram frequency and time-aware transformer, *Comput. Secur.* 128 (C) (May 2023). doi:10.1016/j.cose.2023.103171.
- [15] C. Xi, H. Wang, X. Wang, A novel multi-scale network intrusion detection model with transformer, *Scientific Reports* 14 (1) (2024) 23239.
- [16] G. Apruzzese, P. Laskov, J. Schneider, Sok: Pragmatic assessment of machine learning for network intrusion detection, in: *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, 2023, pp. 592–614. doi:10.1109/EuroSP57164.2023.00042.

- [17] C. Avci, B. Tekinerdogan, C. Catal, Design tactics for tailoring transformer architectures to cybersecurity challenges, *Cluster Computing* 27 (7) (2024) 9587–9613. doi:10.1007/s10586-024-04355-0.
- [18] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, L. Cavallaro, Insomnia: Towards concept-drift robustness in network intrusion detection, in: *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security, AISEC '21*, Association for Computing Machinery, New York, NY, USA, 2021, p. 111–122. doi:10.1145/3474369.3486864. URL <https://doi.org/10.1145/3474369.3486864>
- [19] M. Cantone, C. Marrocco, A. Bria, Machine learning in network intrusion detection: A cross-dataset generalization study, *IEEE Access* 12 (2024) 144489–144508.
- [20] M. Sarhan, S. Layeghy, M. Portmann, Evaluating standard feature sets towards increased generalisability and explainability of ml-based network intrusion detection, *Big Data Research* 30 (2022) 100359.
- [21] L. D. Manocchio, S. Layeghy, M. Gallagher, M. Portmann, An empirical evaluation of preprocessing methods for machine learning based network intrusion detection systems, *Engineering Applications of Artificial Intelligence* 158 (2025) 111289.
- [22] A. Dijk, LSPR25 dataset on Zenodo, reserved DOI (2025). doi:10.5281/zenodo.17093949.
- [23] A. Dijk, E. Halisdemir, C. Melella, A. Schu, M. Pihelgas, R. Meier, Lspr23: A novel ids dataset from the largest live-fire cybersecurity exercise, *Journal of Information Security and Applications* 85 (2024) 103847.
- [24] A. Dijk, R. Meier, C. Melella, M. Pihelgas, R. Vaarandi, V. Lenders, Next steps in cyber blue team automation—leveraging the power of llms, in: *2025 17th International Conference on Cyber Conflict: The Next Step (CyCon)*, 2025, pp. 209–226. doi:10.23919/CyCon65856.2025.11103720.
- [25] R. Hofstede, P. Čeleda, B. Trammell, I. Drago, R. Sadre, A. Sperotto, A. Pras, Flow monitoring explained: From packet capture to data analysis with netflow and ipfix, *IEEE Communications Surveys & Tutorials* 16 (4) (2014) 2037–2064. doi:10.1109/COMST.2014.2321898.
- [26] A. Corsini, S. J. Yang, G. Apruzzese, On the evaluation of sequential machine learning for network intrusion detection, in: *Proceedings of the 16th International Conference on Availability, Reliability and Security, ARES '21*, Association for Computing Machinery, New York, NY, USA, 2021, pp. 1–10. doi:10.1145/3465481.3470065. URL <https://doi.org/10.1145/3465481.3470065>

- [27] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, et al., Toward generating a new intrusion detection dataset and intrusion traffic characterization., *ICISSp 1* (2018) (2018) 108–116.
- [28] G. Engelen, V. Rimmer, W. Joosen, Troubleshooting an intrusion detection dataset: the cicids2017 case study, in: *2021 IEEE Security and Privacy Workshops (SPW)*, IEEE, 2021, pp. 7–12.
- [29] M. Verkerken, L. D’hooge, T. Wauters, B. Volckaert, F. De Turck, Towards model generalization for intrusion detection: Unsupervised machine learning techniques, *Journal of Network and Systems Management* 30 (1) (2022) 12.
- [30] M. Sarhan, S. Layeghy, N. Moustafa, M. Gallagher, M. Portmann, Feature extraction for machine learning-based intrusion detection in iot networks, *Digital Communications and Networks* 10 (1) (2024) 205–216.
- [31] M. Ali, M.-u. Haque, M. H. Durad, A. Usman, S. M. Mohsin, H. Mujlid, C. Maple, Effective network intrusion detection using stacking-based ensemble approach, *International Journal of Information Security* 22 (6) (2023) 1781–1798.
- [32] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: *2010 IEEE symposium on security and privacy*, IEEE, 2010, pp. 305–316.
- [33] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, K. Rieck, Dos and don’ts of machine learning in computer security, in: *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3971–3988.
- [34] P. Rubin-Delanchy, D. J. Lawson, M. J. Turcotte, N. Heard, N. M. Adams, Three statistical approaches to sessionizing network flow data, in: *2014 IEEE Joint Intelligence and Security Informatics Conference*, 2014, pp. 244–247. doi:10.1109/JISIC.2014.46.
- [35] G. Kim, S. Lee, S. Kim, A novel hybrid intrusion detection method integrating anomaly detection with misuse detection, *Expert Systems with Applications* 41 (4) (2014) 1690–1700.
- [36] H. Clausen, G. Grov, D. Aspinall, Cbam: A contextual model for network anomaly detection, *Computers* 10 (6) (2021). doi:10.3390/computers10060079.
- [37] M. Zhong, M. Lin, C. Zhang, Z. Xu, A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges, *Comput. Secur.* 141 (C) (Jun. 2024). doi:10.1016/j.cose.2024.103821.

- [38] G. Apruzzese, L. Pajola, M. Conti, The cross-evaluation of machine learning-based network intrusion detection systems, *IEEE Transactions on Network and Service Management* 19 (4) (2022) 5152–5169.
- [39] D. Pinto, I. Amorim, E. Maia, I. Praça, A review on intrusion detection datasets: tools, processes, and features, *Computer Networks* 262 (2025) 111177.
- [40] J. Casajús-Setién, C. Bielza, P. Larrañaga, Anomaly-based intrusion detection in iiot networks using transformer models, in: *2023 IEEE International conference on cyber security and resilience (CSR)*, IEEE, 2023, pp. 72–77.
- [41] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, *Advances in neural information processing systems* 30 (2017).
- [42] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, Y. Liu, Roformer: Enhanced transformer with rotary position embedding, *Neurocomputing* 568 (2024) 127063. doi:<https://doi.org/10.1016/j.neucom.2023.127063>.
- [43] I. Beltagy, M. E. Peters, A. Cohan, Longformer: The long-document transformer, *arXiv preprint arXiv:2004.05150* (2020).
- [44] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, et al., Big bird: Transformers for longer sequences, *Advances in neural information processing systems* 33 (2020) 17283–17297.
- [45] Hugging Face, Bigbird, Hugging Face Transformers documentation, accessed 2026-03-18 (2026).
- [46] Y. Yang, X. Peng, Bert-based network for intrusion detection system, *EURASIP Journal on Information Security* 2025 (1) (2025) 11. doi:[10.1186/s13635-025-00191-w](https://doi.org/10.1186/s13635-025-00191-w).
- [47] K. Cho, B. Van Merriënboer, Ç. Gulçehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using rnn encoder–decoder for statistical machine translation, in: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014, pp. 1724–1734.
- [48] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [49] Z. Wu, H. Zhang, P. Wang, Z. Sun, Rtids: A robust transformer-based approach for intrusion detection system, *IEEE Access* 10 (2022) 64375–64387. doi:[10.1109/ACCESS.2022.3182333](https://doi.org/10.1109/ACCESS.2022.3182333).

- [50] J. Davis, M. Goadrich, The relationship between precision-recall and roc curves, in: Proceedings of the 23rd international conference on Machine learning, 2006, pp. 233–240.
- [51] T. Saito, M. Rehmsmeier, The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets, PloS one 10 (3) (2015) e0118432.
- [52] R. J. Tibshirani, B. Efron, An introduction to the bootstrap, Monographs on statistics and applied probability 57 (1) (1993) 1–436.
- [53] D. N. Politis, J. P. Romano, The stationary bootstrap, Journal of the American Statistical association 89 (428) (1994) 1303–1313.

Appendix A. Complete list of used flow features

We used the features described in [23]. The exact fields are:

- SrcPort, DstPort, Protocol, Flow Duration, Flow Bytes/s, Flow Packets/s
- Tot Fwd Pkts, Tot Bwd Pkts, Total Length of Fwd Packet, Total Length of Bwd Packet
- Fwd Packet Length Min/Max/Mean/Std, Bwd Packet Length Min/Max/Mean/Std
- Flow IAT Mean/Min/Max/Stddev
- Fwd IAT Min/Max/Mean/Std/Tot
- Bwd IAT Min/Max/Mean/Std/Tot
- Fwd PSH flags, Bwd PSH flags, Fwd URG flags, Bwd URG flags
- Fwd Header Length, Bwd Header Length
- Fwd Packets/s, Bwd Packets/s
- Packet Length Min/Max/Mean/Std/Variance
- FIN Flag Cnt, SYN Flag Cnt, RST Flag Cnt, PSH Flag Cnt, ACK Flag Cnt, URG Flag Cnt, CWR Flag Cnt, ECE Flag Cnt
- Down/Up Ratio, Average Packet Size, Fwd Segment Size Avg, Bwd Segment Size Avg
- Fwd Bytes/Bulk Avg, Fwd Packet/Bulk Avg, Fwd Bulk Rate Avg, Bwd Bytes/Bulk Avg, Bwd Packet/Bulk Avg, Bwd Bulk Rate Avg
- Subflow Fwd Packets, Subflow Fwd Bytes, Subflow Bwd Packets, Subflow Bwd Bytes
- FWD Init Win Bytes, Bwd Init Win Bytes, Fwd Act Data Pkts, Fwd Seg Size Min
- Active Min/Mean/Max/Std, Idle Min/Mean/Max/Std
- L3/L4 Protocol, Int/Ext Dst IP
- External_src, External_dst